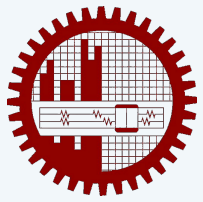


Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

CSE 105 Question Bank

Data Structures & Algorithms

CSE 105 (L-1/T-2)
CSE 203 (L-2/T-1) & CSE 207 (L-2/T-2)

Topics: Asymptotic Analysis, Data Structures, Graph Traversals, Divide & Conquer, Greedy, Dynamic Programming, Sorting & more

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— LaTeX Typesetting
NotebookLM — Research & Extraction
Claude AI — LaTeX Typesetting
ChatGPT — Prompt

Table of Contents

BUET · CSE 105

DSA

Introduction to Algorithms

Intro, Basic Search, Array Mapping

T1 — Introduction to Algorithms

Q1. Consider the code snippet for a binary search, where you are searching for an item **x** from an array **a** of size **n**. If all primitive operations (addition/subtraction, comparison, assignment, etc.) take one unit of time each to execute, what will be the time complexity of the above algorithm by counting all the primitive operations performed. Can you show the worst-case time complexity of the above algorithm using big-Oh notation? **(5 marks)** [CSE 203 | 2019–20]

Algorithm 1 BinarySearch(a, x)

```
1: left ← 0
2: right ← a.size()
3: while left < right do
4:     middle ← (left + right) / 2
5:     if a[middle] ≤ x then
6:         left ← middle + 1
7:     else
8:         right ← middle
9:     end if
10: end while
11: if left > 0 and a[left - 1] == x then
12:     return true
13: else
14:     return false
15: end if
```

Q2. Write down two algorithms for determining whether a year is a leap year. Deduce in detail the average-case complexity of both algorithms. **(5+5 marks)** [CSE 207 | 2006–07, 2013–14]

Q3. Write down two versions of the binary search algorithm. Deduce the average-case complexity of both. **(5+5 marks)** [CSE 207 | 2013–14]

Q4. Write down traditional algorithms for finding the minimum and maximum of an array, and a recursive algorithm. Compare their complexities. **(10 marks)** [CSE 207 | 2013–14]

Q5. What is a data structure? When is a data structure called a linear data structure? Write 4 (four) properties of a linear data structure. **(1.5+1.5+4=7 marks)** [CSE 203 | 2013–14]

Q6. Let `a : array[`$l_1 \,..\, u_1$`, `$l_2 \,..\, u_2$`]` be a 2D integer array. Assume b is the base address of the array `a`. Write the Array Mapping Function to calculate `address(a[i,j])` in:

(a) Column-major order

(b) Row-major order

(5+5=10 marks) [CSE 203 | 2013–14]

Q7. What is a data structure? Write the properties of a linear data structure. **(1+4=5 marks)** [CSE 203 | 2012–13]

Q8. Let `a : array[`$l_1 \,..\, u_1$`, `$l_2 \,..\, u_2$`, `$l_3 \,..\, u_3$`]` be a 3-dimensional array. Assume that b and c are the base address and component length of the array `a` respectively. Write the Array Mapping Function to calculate addr$(a[i, j, k])$. **(10 marks)** [CSE 203 | 2012–13]

Q9. Write down the algorithm for binary search. Analyze its best-case, average-case, and worst-case complexities. Simulate the search of elements 2 and 10 in the array $[1, 2, 4, 7, 9, 11, 12, 13, 14]$ by showing the values of `low`, `high`, `mid`, and `found`. **(5+5+5=15 marks)** [CSE 207 | 2008–09]

Q10. Write down an algorithm for finding the minimum and maximum of an array. Deduce its complexity. **(10 marks)** [CSE 207 | 2007–08]

Q11. Deduce the best-case, worst-case, and average-case complexity of sequential search in both successful and unsuccessful cases. **(10 marks)** [CSE 207 | 2007–08]

Q12. Write down an algorithm for binary search that delays the finding until the end. Deduce its average-case complexity. **(5+10 marks)** [CSE 207 | 2006–07]

BUET · CSE 105

DSA

Asymptotic Notations: $\mathcal{O}$, Ω, Θ, o, ω

Growth of Functions, Notations, Code Complexity

T2 — Asymptotic Notations: $\mathcal{O}$, Ω, Θ, o, ω

Q1. Analyze and compare the growth rate of $n^{\log\log n}$ and $(\log n)^n$. **(10 marks)** [CSE 105 | 2022–23]

Q2. Derive the time complexity of the following code fragments:

(a)
```
for(int i=0; i<n; ++i)
    for(int j=0; j<i; ++j)
      sum += i+j;
```

(b)
```
for(int i=0; i<n; ++i)
    for(int j=0; j<i*i; j++)
      sum += i+j;
```

(c)
```
for(int i=0; i<n*n; i++)
    for(int j=0; j<i; j++)
      sum += i+j;
```

(d)
```
for(int k=1; k<=n; k*=2)
    for(int j=1; j<=k; j++)
      sum++;
```

(4×4=16 marks) [CSE 105 | 2021–22]

Q3. In an array-based implementation of a stack, when the array space runs out, the array capacity is increased. Analyze and explain the following scenarios through an amortized analysis of the `push()` function:

(a) Increasing the capacity of the array by 1

(b) Doubling the capacity of the array

(2×7=14 marks) [CSE 105 | 2021–22]

Q4. Explain what the three figures (Figure 1(i)–(iii)) signify with respect to asymptotic notations. **(– marks)** [CSE 105 | 2021–22]

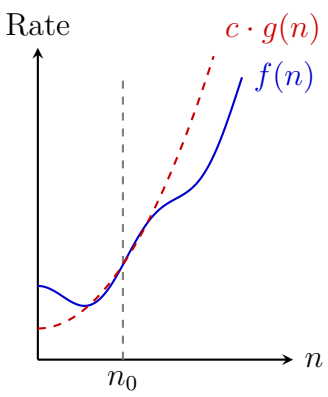


Figure : (i)

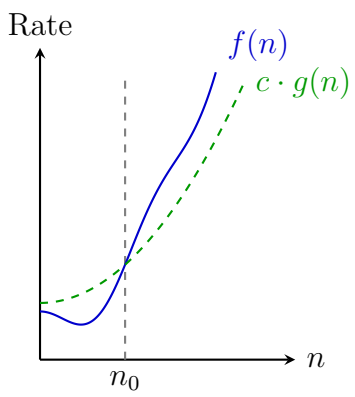


Figure : (ii)

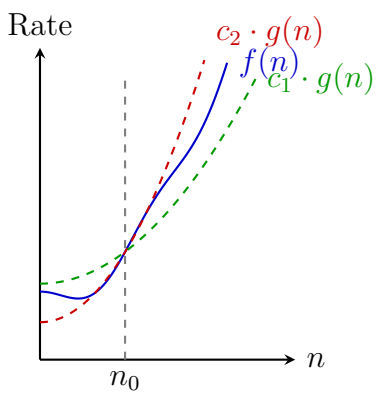


Figure : (iii)

Q5. Find a tight upper bound on the complexity of the following code segments:

(a)
```
void function(int n) {
        int count = 0;
        for (int i=n/2; i<=n; i++)
            for (int j=1; j+n/2<=n; j++)
                for (int k=1; k<=n; k = k*2)
                    count++;
    }
```

(b)
```
void function(int n) {
        int count = 0;
        for (int i=n/2; i<=n; i++)
            for (int j=1; j<=n; j = 2*j)
                for (int k=1; k<=n; k = k*2)
                    count++;
    }
```

(10 marks) [CSE 203 | 2021–22]

Q6. Deduce the time complexity of the following code fragments:

(a)
```
k = 1;
while (n > 1) {
    n = n/2;
    k++;
}
```

(b)
```
sum = 0;
for (k=1; k<=n; k*=2)
    for (j=1; j<=k; j++)
        sum++;
```

(2×6=12 marks) [CSE 203 | 2020–21]

Q7. Identify whether the following statements are true or false, clearly justifying your answer:

(a) If an algorithm has time complexity $O(n^2)$, then it always makes n^2 steps.

(b) An algorithm with time complexity $O(n)$ is always slower than an algorithm with time complexity $O(\log n)$, for any input.

(c) An algorithm that makes $c_1 \log_2 n$ steps and an algorithm that makes $c_2 \log_{10} n$ steps are in the same complexity class (c_1, c_2 are constants).

(d) An $O(2^n)$ algorithm can never be faster than an $O(n)$ algorithm.

(12 marks) [CSE 203 | 2020–21]

Q8. What does the term *asymptotic* mean in complexity analysis? Can we show the worst-case and average-case time complexity using Big-Θ notation? If yes, how and when? **(5+5+5 marks)** [CSE 203 | 2019–20]

Q9. Derive the time complexity of the following code snippet and write down the asymptotic complexity using Big-O notation. Can we show the complexity using Big-Θ notation as well? What is the difference in meaning?

```
int i, j, k = 0;
for (i = n/2; i <= n; i++) {
    for (j = 2; j <= n; j = j*2) {
        k = k + n/2;
    }
}
```

(10 marks) [CSE 203 | 2019–20]

Q10. Express the following function in Θ notation with proper explanation:

$$f(n) = 5n - 2n^2$$

(8 marks) [CSE 203 | 2019–20]

Q11. Suppose that you observe that a program takes 30 seconds to complete on inputs of size 600, 4.5 minutes on inputs of size 1800, and 20 minutes on inputs of size 2000. Develop a reasonable assumption for the order of growth of the running time. **(10 marks)** [CSE 203 | 2018–19]

Q12. What is the difference between Big-O (O) and Little-o (o) asymptotic notations? Find appropriate $f(n)$ and $g(n)$ (if any) for each of the following cases:

(a) $f(n) = O(g(n))$, but $g(n) \neq O(f(n))$

(b) $f(n) = O(g(n))$, and $f(n) = \Omega(g(n))$

(c) $f(n) = \Omega(g(n))$, and $f(n) = \omega(g(n))$

(d) $f(n) = \Theta(g(n))$, and $f(n) = o(g(n))$

(5+10 marks) [CSE 203 | 2017–18]

Q13. Define Big-O, Ω, and Θ notations. Arrange the following functions in ascending order of growth rate (i.e., if $f(n)$ precedes $g(n)$, then $f(n) = O(g(n))$). Provide justifications.

$$f_1 = n, \quad f_2 = \log_2 n, \quad f_3 = 2^{\sqrt{\log_2 n}}, \quad f_4 = n \ln n, \quad f_5 = 2^n$$

(6+9 marks) [CSE 203 | 2016–17]

Q14. Prove that $15n^4 + 13n^2 + 11n = O(n^4)$, but $15n^4 + 13n^2 + 11n \neq O(n^2)$. **(10 marks)** [CSE 203 | 2015–16]

Q15. For each of the following pairs indicate whether $f(n)$ is $O(g(n))$, $\Omega(g(n))$ or both (i.e. $\Theta(g(n))$). Provide brief justifications

$f(n)$	$g(n)$
$\sqrt{n}$	$\log_2 n$
$n^{1.01}$	$n \log_2^2 n$
2^n	2^{n+1}
$n!$	2^n
n^{100}	1.01^n
$\sum_{i=1}^{n} i^k$	n^{k+1}

(6+9=15 marks) [CSE 207 | 2015–16]

Q16. What is asymptotic analysis of algorithms? Define Big-O, Ω, and Θ notations. For the functions $f(n) = n \log n$ and $g(n) = 10n \log_{10} n$, show whether f is $O(g)$, or f is $\Omega(g)$, or f is $\Theta(g)$. **(3+6+8 marks)** [CSE 207 | 2014–15]

Q17. Prove that $5n^3 + 3n = O(n^4)$, but $5n^3 + 3n \neq O(n^2)$. **(5+5=10 marks)** [CSE 203 | 2012–13]

Q18. Prove that $O(n^3) \cdot O(n) = O(n^4)$. **(5 marks)** [CSE 203 | 2011–12]

Q19. Write the main three reasons why we usually concentrate on finding the worst-case running time of an algorithm. **(5 marks)** [CSE 203 | 2011–12]

Q20. Why is $n/2 \neq o(n)$? **(5 marks)** [CSE 207 | 2009–10]

Q21. Write down an algorithm for searching z sequentially in array A with n elements. Discuss its average-case, worst-case, and best-case complexities. How can this algorithm be improved? **(5+10+5=20 marks)** [CSE 207 | 2008–09]

Q22. Define big O-notation, Ω-notation and Θ-notation.

BUET · CSE 105

DSA

Correctness Proof & Algorithm Analysis

Master Theorem, Recurrence, Substitution, Loop Invariant

T3 — Correctness Proof & Algorithm Analysis

Q1. Analyze the following recurrences using the Master Theorem:

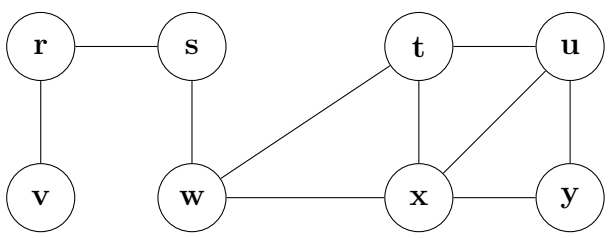
(a) $T(n) = 2T(n/2) + n \log n$

(b) $T(n) = 2T(n/2) + n^2$

(5+5=10 marks) [CSE 105 | 2023–24]

Q2. Using the Master Theorem, prove that Strassen's algorithm improves the runtime of multiplying two $n \times n$ matrices compared to the naive approach with time complexity $O(n^3)$. **(10 marks)** [CSE 105 | 2023–24]

Q3. State the correctness theorem of Breadth-First Search (BFS). Use this theorem to judge whether the value $u.d$ assigned to a vertex u in BFS depends on the order of appearance of the vertices in the corresponding adjacency list. Demonstrate with an example graph whether the breadth-first tree produced by BFS depends on the ordering of vertices in the adjacency lists if the same source vertex is used in every ordering. **(17 marks)** [CSE 105 | 2022–23]



Q4. "It is asymptotically faster to square an n-bit integer than to multiply two n-bit integers." Do you agree with this statement? Justify your answer. **(10 marks)** [CSE 203 | 2021–22]

Q5. Using the substitution method, solve the following recurrence:

$$T(n) = 4T\left(\frac{n}{2}\right) + 100n^2$$

(11 marks) [CSE 203 | 2020–21]

Q6. Explain whether we can apply the Master Method to any recurrence of the form $T(n) = aT(n/b) + f(n)$. If not, explain a case where the theorem cannot be applied. **(8 marks)** [CSE 203 | 2019–20]

Q7. Traditional insertion sort uses a linear search to scan (backward) through the sorted subarray $A[1 \ldots j-1]$, where $A[j]$ is the element to be inserted. Prove or disprove the claim that we can use binary search instead to improve the overall worst-case running time of insertion sort to $\Theta(n \log n)$. **(8 marks)** [CSE 203 | 2019–20]

Q8. Prove or disprove the following: if we choose the element at the middle index as pivot during partition in Quicksort, the algorithm will never have a runtime of $\Theta(n^2)$. **(10+8 marks)** [CSE 203 | 2019–20]

Q9. Consider a divide-and-conquer algorithm: if the problem size is ≤ 5, it solves the problem in constant time; otherwise, it divides the problem of size n into 4 subproblems each of size $n/3$, solves them recursively, and merges in $O(n)$ time. The division itself takes $O(\sqrt{n} \log n)$ time. Write the recurrence relation for $T(n)$ and solve the following recurrences using the Master Theorem:

(a) $T(n) = 9T(n/3) + 5n + n^2 + 8 \log n$

(b) $T(n) = 3T(n/4) + 3n^2 + 4n + 5$

(c) $T(n) = 4T(n/2) + 5n^3 + 7n$

(10 marks) [CSE 203 | 2018–19]

Q10. Suppose you are choosing among three algorithms for multiplying two $n \times n$ matrices:

(a) Algorithm A (Strassen's) divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes seven recursive calls, and combines them in $\Theta(n^2)$ time.

(b) Algorithm B divides each matrix into twelve $n/3 \times n/3$ matrices, makes nine recursive calls, and combines them in $\Theta(n^2)$ time.

(c) Algorithm C first multiplies two $(n-1) \times (n-1)$ matrices, then combines the result with two vectors of size n in $\Theta(n^2)$ time.

Determine the asymptotic running time of each algorithm (use the Master Theorem where applicable) and determine which algorithm you would choose. **(15 marks)** [CSE 203 | 2016–17]

Q11. Suppose you are choosing among three algorithms for multiplying two $n \times n$ matrices:

(a) Algorithm A divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes 8 recursive calls, and combines in $\Theta(n^2)$ time.

(b) Algorithm B (Strassen's) divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes 7 recursive calls, and combines in $\Theta(n^2)$ time.

(c) Algorithm C divides each $n \times n$ matrix into nine $n/3 \times n/3$ matrices, makes 9 recursive calls, and combines in $\Theta(n^2)$ time.

Use the Master Theorem to determine asymptotic running times of all three algorithms and choose the best one. **(15 marks)** [CSE 207 | 2015–16]

Q12. State the Master Theorem. Explain why the Master Method cannot be used to solve the recurrence $T(n) = 2T(n/2) + n \log n$. Using the substitution method, prove that $T(n)$ is $O(n \log^2 n)$. **(6+6+6 marks)** [CSE 207 | 2014–15]

Q13. Using the Master Method, prove that the recurrence $T(n) = 3T(n/4) + n \lg n$ solves to $T(n) = \Theta(n \lg n)$. **(12 marks)** [CSE 207 | 2009–10]

Q14. Write the pseudocode of Strassen's algorithm for multiplying two $n \times n$ matrices. Derive the running time. **(8+10=18 marks)** [CSE 207 | 2009–10]

BUET · CSE 105

DSA

Arrays, Linked Lists, Stacks & Queues

Linear Data Structures, Implementation, Operations

T4 — Arrays, Linked Lists, Stacks & Queues

Q1. You are asked to merge two singly linked lists whose elements are sorted in ascending order. The output list will also be sorted in ascending order. Sketch an algorithm to solve the problem. Analyze that your algorithm runs in linear time on the number of elements of the output list. **(15 marks)** [CSE 105 | 2023–24]

Q2. You are given two linked lists of unsorted elements. Investigate whether you can develop a constant time algorithm to concatenate them if both of the lists are (i) singly linked lists (SLL), (ii) doubly linked list (DLL), (iii) circular SLL, (iv) circular DLL. If yes, sketch the algorithm and examine its correctness; otherwise demonstrate why a constant time algorithm is not possible. **(20 marks)** [CSE 105 | 2023–24]

Q3. Assume that a string contains only left and right parentheses which may or may not be balanced and properly nested. For example, the string ``(((()))()'' contains properly nested pairs of parentheses, but the string ``)()(' does not, and the string ``())'' does not contain properly matching parentheses. Using a stack, construct an algorithm that returns `true` if a string contains properly nested and balanced parentheses, and `false` otherwise. Modify the algorithm so that it returns the position in the string of the first offending parenthesis if the string is not properly nested and balanced — if an excess right parenthesis is found, return its position; if there are too many left parentheses, return the position of the first excess left parenthesis. Return -1 if the string is properly balanced and nested. Assume indices start from 0. **(25 marks)** [CSE 105 | 2023–24]

Q4. Given an integer k and a queue of integers, how do you reverse the order of the first k elements of the queue, leaving the other elements in the same relative order? For example, if $k = 4$ and the queue has the elements $[10, 20, 30, 40, 50, 60, 70, 80, 90]$, the output should be $[40, 30, 20, 10, 50, 60, 70, 80, 90]$. **(10 marks)** [CSE 105 | 2023–24]

Q5. Implement a Circular Queue using an array. Include: `enqueue(x)`, `dequeue()`, `is_empty()`, `is_full()`, `front()`. **(15 marks)** [CSE 105 | 2022–23]

Q6. Write a function to delete a node with a given key from a doubly linked list (unique keys, access to both `head` and `tail`). **(10 marks)** [CSE 105 | 2022–23]

Q7. How would you reverse a singly linked list (without duplicating the list)? Write the steps (pseudocode) conceptually. **(4 marks)** [CSE 105 | 2022–23]

Q8. How can you remove duplicate values from a sorted singly linked list? **(5 marks)** [CSE 105 | 2022–23]

Q9. How can you check if a singly linked list is a palindrome? Write the steps involved conceptually. **(10 marks)** [CSE 105 | 2022–23]

Q10. Given an array of integers (with both negative and positive numbers), write a program to find the K-th largest sum of a contiguous subarray within the array. **(10 marks)** [CSE 203 | 2021–22]

Q11. Suppose the numbers $0, 1, 2, \ldots, 9$ were pushed onto a stack in that order, but pops occurred at random points between the various pushes. Determine which of the following two sequences is a valid pop sequence and explain why:

$$S_1 : 3, 2, 6, 5, 7, 4, 1, 0, 9, 8$$

$$S_2 : 3, 2, 6, 4, 7, 5, 1, 0, 9, 8$$

(3 marks) [CSE 105 | 2021–22]

Q12. A stack is being used to parse matching parentheses, brackets, and braces — `(`, `[`, and `{`. Show the state of the stack at the end of the given code fragment. **(10 marks)** [CSE 105 | 2021–22]

```cpp
#include <iostream>
using namespace std;

int main()
{
    const int M = 30;
    int N = 1024;
    int parent[N], height[N], depth[N];
    int maxheights[100];
    for (int i = 0; 1 < 100; ++i)
    {
        maxheights[i] = 0;
    }
    double minmeandepth = 1e300;
    double meandepths = 0;
    double maxmeandepth = 0;
    for (int j = 0; j < M; ++j)
    {
    for (int i = 0; i < N; ++i)
    {
        parent[i] = -1;
        height[i] = 0;
```

```
23      }
24      for (int i = 0; i < N - 1; ++i)
25      while (true)
26      {
27      int p1 = rand() & (N - 1);
28      int p2 = rand() & (N - 1);
29
30      int s1 = pi1;
31      int s2 = p2;
32
33      while ( parent[
```

Q13. You are given access to the following functions of a list data structure:

- `init(n)` — Initialize an array $L[0..n-1]$ of length n for integers. Current position 'points to' $L[0]$. All the functions below are applied on this array which is not accessible directly.
- `insert(item)` — Inserts an element at the current position and current position is 'incremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). Here, *item* is the element to be inserted. In case the list is full, it returns ERR (a large negative constant).
- `remove()` — Removes the element at the current position and current position is 'decremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). In case the list is empty, it returns ERR (a large negative constant).
- `value()` — Returns the value of the element at the current position. Current position remains unchanged. In case the list is empty, it returns ERR (a large negative constant).
- `moveToStart()` — Sets the current position to $L[0]$.
- `moveToEnd()` — Sets the current position to $L[n-1]$.
- `next()` — Increments the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). If $k = n-1$, it returns ERR (a large negative constant).
- `prev()` — Decrements the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). If $k = 0$, it returns ERR (a large negative constant).

Now you have to build one queue and one stack in one List data structure as defined above. The queue should grow from the beginning of the list (i.e., first element of the queue should be at $L[0]$) and the stack should grow from the end of the list (i.e., the first element of the stack should be at $L[n-1]$). You have no access to the code of the above implementation and can only call these functions in your code. You need to write appropriate codes (no specific programming language is required; pseudo code is fine) for implementing the queue and stack simultaneously as described above. Please explain your code by giving appropriate comments. In particular, you need to implement the following four functions:

- `push(item)` — push a positive integer, *item* in the stack. If there is an error (e.g., no more space available for growing the stack in the list), it should output an appropriate error message and return with -1.
- `pop()` — pop an element from the stack. If there is an error, it should output an appropriate error message and return with -1.
- `enqueue(item)` — enqueue a positive integer, *item* in the queue. If there is an error (e.g., no more space available for growing the queue in the list), it should output an appropriate error message and return with -1.
- `dequeue()` — dequeue an element from the queue. If there is an error, it should output an appropriate error message and return with -1.

(25 marks) [CSE 105 | 2021–22]

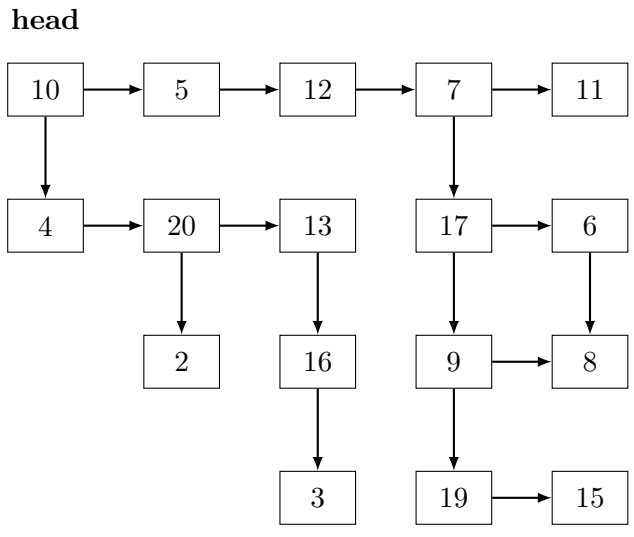
Q14. Convert a Binary Tree to a Circular Doubly Linked List using inorder traversal (left/right pointers become prev/next). **(15 marks)** [CSE 203 | 2021–22]

Q15. Create `KStacks`: two stacks in a single array efficiently. Implement push1,pop1, push2,pop2. **(10 marks)** [CSE 203 | 2021–22, 2016–17]

Q16. Given a string consisting of opening and closing parentheses, find the length of the longest valid parenthesis substring. **(10 marks)** [CSE 203 | 2021–22]

Q17. Given a Queue data structure that supports `enqueue()` and `dequeue()`, implement a Stack using only instances of Queue and Queue operations. **(10 marks)** [CSE 203 | 2021–22]

Q18. Given a linked list where in addition to the next pointer, each node has a child pointer, which may or may not point to a separate list. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure, as shown in Figure . You are given the head of the first level of the list. Flatten the list so that all the nodes appear in a single-level linked list. You need to flatten the list in a way that all nodes at the first level should come first, then nodes of second level, and so on.



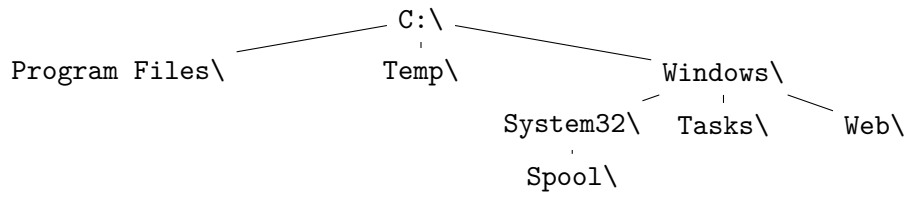
The above list should be converted to $10 \to 5 \to 12 \to 7 \to 11 \to 4 \to 20 \to 13 \to 17 \to 6 \to 2 \to 16 \to 9 \to 8 \to 3 \to 19 \to 15$. **(15 marks)** [CSE 203 | 2021–22]

Q19. Write a program to reverse a stack using recursion, without using any loop. **(10 marks)** [CSE 203 | 2021–22]

Q20. Why is Binary Search preferred over Ternary Search? **(10 marks)** [CSE 203 | 2021–22]

Q21. Write a code snippet to implement a queue using a circular array, where each element contains ID, name, and department of a student. **(13 marks)** [CSE 203 | 2019–20]

Q22. Consider the following hierarchical structure of a file system. Write a program to print the directory hierarchy of the file system, where all children of a parent will be placed one tab-space right from the parent. Use depth-first tree traversal technique to perform the above task.



(12 marks) [CSE 203 | 2019–20]

Q23. Give the algorithm for converting an infix expression to postfix, and for evaluating a postfix expression using a stack. **(10 marks)** [CSE 203 | 2018–19]

Q24. Ternary search, like binary search, is a divide-and-conquer algorithm. It is mandatory for the array (in which you will search for an element) to be sorted before you begin the search. In this search, after each iteration it neglects $\frac{2}{3}$ part of the array and repeats the same operations on the remaining $\frac{1}{3}$. Write an algorithm for ternary search and show its complexity. **(12 marks)** [CSE 203 | 2018–19]

Q25. Describe a situation where storing items in an array is clearly better than storing items in a linked list. **(8 marks)** [CSE 203 | 2018–19]

Q26. Write two versions of binary search. Simulate on $[23, 56, 8, 10, 12, 15, 20, 21, 23]$ for keys $1, 6, 23, 24$. **(10+10=20 marks)** [CSE 203 | 2017–18]

Q27. How do we cope with hard problems? Discuss. **(10 marks)** [CSE 207 | 2017–18]

Q28. Given a singly linked list and integers m and k, find the node with value m then delete the next k nodes. **(10 marks)** [CSE 203 | 2016–17]

Q29. Write `enqueue` and `dequeue` functions for a FIFO queue implemented using a singly linked list. **(10 marks)** [CSE 203 | 2016–17]

Q30. Deduce the condition under which an array-based implementation of a list would be better in space utilisation than a linked-based implementation. **(10 marks)** [CSE 203 | 2015–16]

Q31. Given input string x, check whether x is of the form $w\#w'$, where w' is the reverse of w. Suggest a data structure and show how to solve it efficiently. **(10 marks)** [CSE 203 | 2015–16]

Q32. Given input k and n, return the key of the n-th node before the one containing k. (All keys distinct.) **(15 marks)** [CSE 203 | 2015–16]

Q33. Your group proposes that the beginning (index 0) of the array be the top of the stack. A friend claims this is inefficient. Do you agree? Justify. **(8 marks)** [CSE 203 | 2015–16]

Q34. Discuss the Drifting Queue Problem with an illustrative example. Propose a solution. Address empty/full recognition. **(5+5+5+4=19 marks)** [CSE 203 | 2015–16]

Q35. What is an application of prefix sums? What are prefix sums of an array $A[i]$, $1 \leq i \leq n$? Write a parallel algorithm that finds prefix sums of the array A. Illustrate for $[15, 35, 40, 25, 20, 50, 10, 30]$ showing bottom-up and top-down traversals. **(15 marks)** [CSE 207 | 2015–16]

Q36. Implement `void delete(int N)` that deletes all Mth nodes such that $M \bmod N = 0$ from the doubly linked list (head is the 1st node). **(15 marks)** [CSE 203 | 2014–15]

Q37. Compare worst-case run times of ArrayList and LinkedList (doubly, with head and tail) for the listed operations. **(4×3=12 marks)** [CSE 203 | 2014–15]

Q38. Give two advantages and disadvantages of using array-based lists over linked lists. **(8 marks)** [CSE 203 | 2014–15]

Q39. Implement a **stack** using a singly linked list (`push`, `pop`, `size`). Only `head` and `tail` pointers; no other global/static variables. **(15 marks)** [CSE 203 | 2014–15]

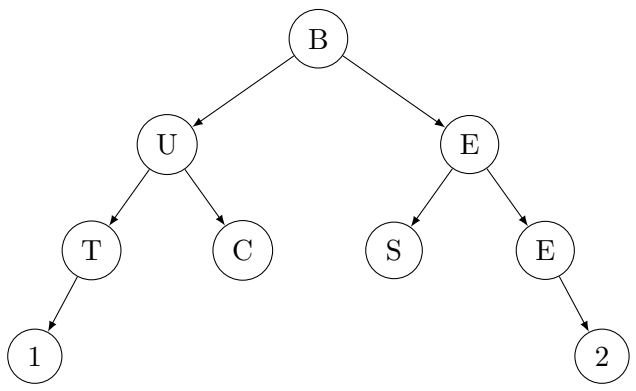
Q40. Suppose we double the memory of an array-based list every time it is full during insertion. Show the average time per insertion for copying is constant. **(12 marks)** [CSE 203 | 2014–15]

Q41. Give two reasons why we should use restrictive data structures (e.g., Stack, Queue) instead of general-purpose ones (e.g., List). **(4 marks)** [CSE 203 | 2014–15]

Q42. Given a doubly linked list and an integer n, write a function `deleteLastNElements()` that deletes the last n nodes from the given linked list. **(8 marks)** [CSE 203 | 2013–14]

Q43. How do you implement a Queue using a Stack? What are the running times of `enqueue` and `dequeue` operations of such an implementation? **(8+2=10 marks)** [CSE 203 | 2013–14]

Q44. Which type of traversal will traverse the binary tree `BUETCSE12` (as shown in figure)? Write an algorithm for such traversal of a binary tree using a Stack or Queue. **(2+10=12 marks)** [CSE 203 | 2013–14]



Q45. Give 2 (two) real-life applications each for: (i) Stack, (ii) Queue, (iii) Priority Queue. **(2+2+2=6 marks)** [CSE 203 | 2013–14]

Q46. Write pseudocodes for `enqueue` and `dequeue` operations for the array implementation of a circular queue. **(7 marks)** [CSE 203 | 2013–14]

Q47. Give a comparison of array and linked list. **(6 marks)** [CSE 203 | 2013–14]

Q48. Write down a linear-time algorithm for finding the maximum sum of a consecutive subsequence of an array. **(15 marks)** [CSE 207 | 2013–14]

Q49. Write two functions for the insertion and deletion operations in a circular linked list. **(5+5=10 marks)** [CSE 203 | 2012–13]

Q50. Explain the advantages and disadvantages of arrays and linked lists. **(5 marks)** [CSE 203 | 2011–12]

Q51. Write pseudocodes for the `Insert` and `Delete` operations of a circular queue. **(6 marks)** [CSE 203 | 2011–12]

Q52. Write down a linear-time algorithm for computing the maximum sum of a consecutive subsequence. Simulate the algorithm on the following array, showing values of `maxsum` and `suffixmax`:

$$2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2, -3$$

(15 marks) [CSE 207 | 2007–08]

Q53. Suppose you are given an implementation of a list Abstract Data Type (ADT) which has the functions listed in Table A. Implement a stack which has the functionality shown in Table B. In your implementation, you can only use the list implementation of Table A. Assume that the elements are integers.**(15 marks)**

Q54. Your friend uses your above stack implementation regularly. A bug has been reported for the `max()` function of your stack implementation. Now your friend needs to implement a separate independent `max()` function which uses the stack data structure (avoiding its own `max()` function). He does not have access to the list data structure of Table A. How will he implement the `max()` function? He cannot use any data structure other than your stack implementation. **(10 marks)**

Q55. We use a doubly linked list (DLL) to keep records of some books. In the list, all the books that belong to the same publisher are placed in consecutive nodes. Now, write a function `addBook(title, publisher)` that inserts a book into the DLL as the first book for its publisher. So, if the list already has three books for a particular publisher, the 4^{th} book will be inserted in front of the three books already in the DLL. For a new publisher, the book should be inserted at the end of the DLL. Assume that each DLL node has 'prev' and 'next' pointers and stores the title and publisher of a book as Strings. **(10 marks)**

Q56. Suppose Figure 8.b-I gives a snapshot of the memory at one point. You are running your program for a linked list based list implementation. In your implementation each node has two variables: an integer *key* and a pointer *ptr*. The current situation of the list is illustrated in Figure 8.b-II. You have further implemented the concept of freelist so as to minimize the number of calls to 'new' and 'delete'. The first, second, third and fourth (i.e., last) node correspond to address 1, 2, 3 and 4 in the memory respectively. Suppose at this point a delete operation is done for the node containing 29 followed by two append operations of two nodes containing values 1111 and 2222 respectively.

Address	Values at Memory Location
0	−31
1	12
2	14
3	29
4	100
5	112
6	4
...	...

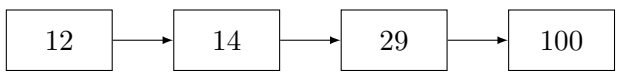


Figure 8.b-II

(i) Draw the updated list after each operation.

(ii) Provide the value of *ptr* variable for each node in Figure 8.b-II before and after the delete operation. If a node remains in the memory after your deletion operation, you would need to provide the value of the *ptr* variable of that node as well.

(iii) Provide the updated values of each memory cell after the two operations (delete and append) are complete.

You can assume that the next call of C++ 'new' operator will return the address 5. **(5+10+10=25 marks)**

BUET · CSE 105

DSA

Trees & Tree Traversals

Binary Trees, General Trees, Traversal Algorithms

T5 — Trees & Tree Traversals

Q1. Analyze the maximum and minimum number of nodes in a binary tree of height h. **(10 marks)** [CSE 105 | 2023–24]

Q2. Prove the following theorems that describe the properties of perfect binary trees:

(a) A perfect tree has $2^{h+1} - 1$ nodes.

(b) The average depth of a node is $\Theta(\ln n)$.

(4+6=10 marks) [CSE 105 | 2022–23]

Q3. You are given a tree of size n as an array `parent[0..n-1]`, where every index i represents a node and `parent[i]` is its immediate parent (-1 for the root). Write an algorithm to find the height of the generic tree.

```
Input: parent[] = {-1, 0, 0, 0, 3, 1, 1, 2}   Output: 2
```

(15 marks) [CSE 203 | 2021–22]

Q4. Write two procedures for finding the successor element and the predecessor element in a binary search tree. Write the preorder, inorder, and postorder traversal sequences of the binary tree whose Euler tour traversal sequence is `a e h i h e a c b f b g b c d j d c a`. Also draw the binary tree. **(10+15 marks)** [CSE 203 | 2016–17]

Q5. Suppose the tree is an unordered tree (i.e., the order of children is not relevant). Determine whether each of the following breadth-first traversals is valid or not, with justification:

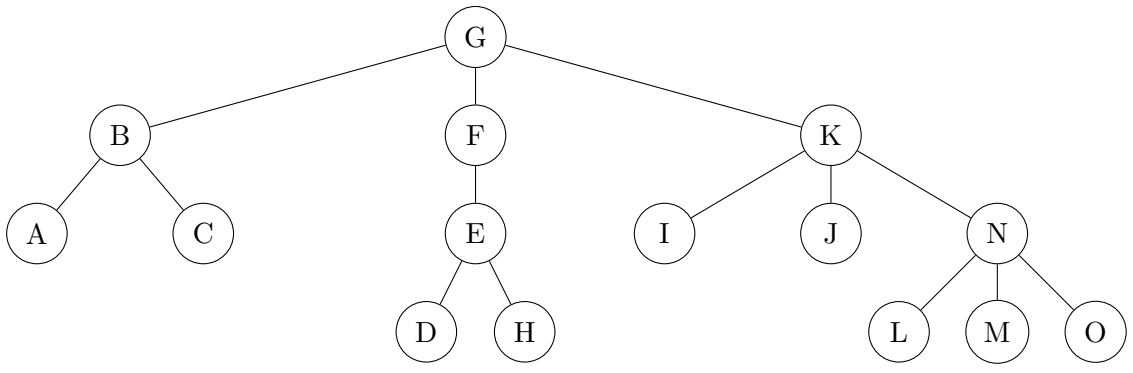
(a) `G F B K E C A I N J D H M L O`

(b) `G F B K E C A I N J D H M L O`

(c) `G K F B I J N E A C O M L H D`

(d) `G F B K E C A I N J O M L H D`

(16 marks) [CSE 203 | 2015–16]



Q6. Suppose that we are given the following definition of a rooted binary tree (**not binary search tree**):

```
struct treeNode
{
    int item ;               //stores the value
    struct treeNode * left ; //points to left child
    struct treeNode * right ;//points to right child
} ;
struct treeNode * root ; //global variable to store tree root node
```

Implement the following function *clone* (**must be recursive**) that makes a copy of a binary tree (keeping the original tree intact). The function will be called using the original root pointer as input argument. The function should return the root node of the new copy. For example calling $x = clone(root)$ will return the new root (of the tree copy) into variable x.

```
struct treeNode * clone(struct treeNode * node);
```

(12 marks) [CSE 203 | 2014–15]

Q7. What is a proper binary tree? Construct the proper binary tree whose inorder and postorder traversals are given as:

- **Inorder:** `BUETCSE12`
- **Postorder:** `BEUCTES21`

(2+8=10 marks) [CSE 203 | 2013–14]

Q8. Draw a binary tree whose preorder and inorder traversals both yield `BUETCSE12`. **(7 marks)** [CSE 203 | 2013–14]

Q9. What is a tree? Write a linear-time algorithm for finding the height of a tree. Analyze the time complexity of the algorithm. **(1+4+4=9 marks)** [CSE 203 | 2012–13 , 2011–12]

Q10. Let T be a proper binary tree with height h. Prove that the number of internal nodes in T is at least h and at most $2^h - 1$. **(4+4=8 marks)** **[CSE 203 | 2012–13 , 2011–12]**

Q11. Construct the binary tree whose in-order and post-order traversals are given as:

- **Inorder:** `i f d j g c a b k h m e`
- **Postorder:** `i j g d f k m h e b a c`

Draw the linked structure for the resulting binary tree. **(10+6=16 marks)** **[CSE 203 | 2012–13]**

BUET · CSE 105

DSA

Heaps & Binary Search Trees

BST Operations, Max/Min Heap, Priority Queue, Build-Heap

T6 — Heaps & Binary Search Trees

Q1. Show the binary search tree (BST) that results from inserting the values 15, 20, 25, 18, 16, 5, and 7 (in that order). Write the enumerations of the tree that result from pre-order, in-order, and post-order traversal of the tree. Write a recursive algorithm that takes as input the pointer to the root of a BST and a value X, and returns `true` if the value X appears in the tree and `false` otherwise. Modify the algorithm if the tree is a generalized binary tree. In both cases, analyze the time complexity of your algorithms. **(25 marks)** [CSE 105 | 2023–24]

Q2. Assume you are given a set of integer numbers. Design a data structure and associated algorithms that support finding the minimum and maximum of the numbers in constant time, and insertion, deletion of maximum, and deletion of minimum in $O(\log n)$ time. *Hint: use two different heaps on the same set of numbers; however, remember their mutual dependencies.* **(15 marks)** [CSE 105 | 2023–24]

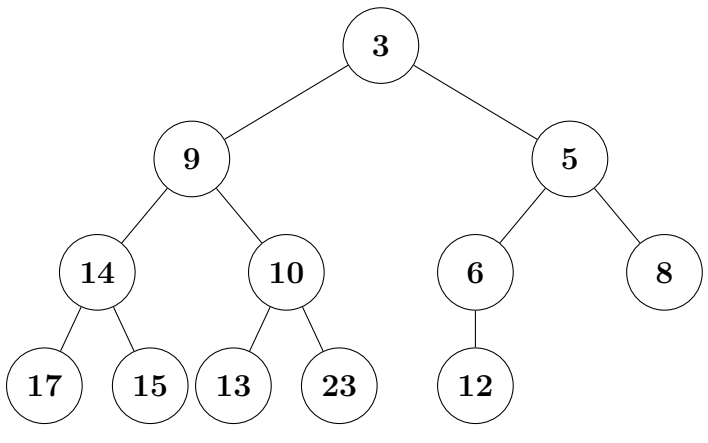
Q3. The key at index 6 of the following max priority queue has been increased to 32. Re-establish the max-heap property of the queue and illustrate the steps using the array only. Justify whether you can use the `max_heapify` algorithm to re-establish the heap property.

1	2	3	4	5	6	7	8	9	10	11	12	13
25	23	20	15	18	16	12	14	7	8	11	4	14

(12 marks) [CSE 105 | 2022–23]

Q4. Given two max-priority queues represented by two arrays, write an efficient algorithm to merge them into a single max-priority queue. Analyse the time complexity of your algorithm. **(17 marks)** [CSE 105 | 2022–23]

Q5. Write a code snippet to traverse the binary tree in an order so that you can store it in an array. Show the placement of the node numbers of the given tree in an array. Write a code to access the parent of a given node. What will be the index no of the array for node 12?**(10+3+8+4=25 marks)** [CSE 105 | 2022–23]



Q6. You are given the following numbers to insert into an empty Binary Search Tree (BST): $5, 7, 8, 12, 15, 27$. Determine which insertion order below would yield the tree with the least height:

(a) $15, 5, 27, 8, 7, 12$

(b) $12, 7, 15, 27, 5, 8$

(c) $8, 27, 7, 5, 15, 12$

(d) $7, 5, 12, 8, 15, 27$

(10 marks) [CSE 105 | 2021–22]

Q7. A perfect balanced BST requires all interior nodes to contain two children and all leaf nodes to be on the same level. A perfect balanced BST of $h = 0$ would have $n = 1$ node, $h = 1$ would have $n = 3$ nodes, $h = 2$ would have $n = 7$ nodes, and so on. Providing appropriate arguments, derive the worst-case running time, in terms of n, for successfully searching for an element in a perfect balanced BST. **(10 marks)** [CSE 105 | 2021–22]

Q8. Present a recursive method `countInRange(root, low, high)`, without any helper functions, to count in a BST all keys that are within a given range. Assume keys in the BST are unique (no duplicates). For example, if the keys in a BST are $4, 7, 10, 14, 15, 17, 20, 30$, then `countInRange(root, 8, 17)` returns 4 and
`countInRange(root, 21, 29)` returns 0. For your convenience a code snippet in C++is given below , but you need not follow any particular programming language . **(10 marks)** [CSE 105 | 2021–22]

```
public class BSTNode {
    int key;
    Value value;
    BSTNode left;
    BSTNode right;
    ...
}
// counts number of keys in BST that are in the range low to high, inclusive
// i.e. all keys k such that low <= k <= high
public static int countInRange(BSTNode root, int low, int high) {
```

```
        // COMPLETE THIS METHOD
}
```

Q9. Present an algorithm to delete a full node in a binary search tree. Explain how it works with illustrative examples. **(10 marks)** [CSE 105 | 2021–22]

Q10. Consider the following array A of integers:(index start from 1)

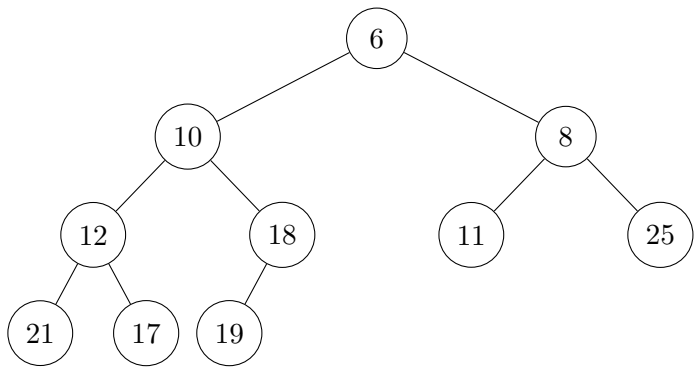
$$A = [16,\ 4,\ 10,\ 7,\ 14,\ 9,\ 3,\ 2,\ 8,\ 1]$$

Verify whether the array is a max heap. Derive the complexity of your verification algorithm. Justify whether the algorithm `max_heapify(A, i)` will establish the max heap property of the array if applied with index $i = 2$. Analyze the time complexity of the `max_heapify` algorithm. **(20 marks)** [CSE 105 | 2021–22]

Q11. Calculate the number of swap operations (data interchanges) required if you apply the `Build_Max_Heap` algorithm on the array $A = \{5, 3, 17, 10, 84, 19, 6, 22, 9\}$. Justify whether the max heap produced can be used as a priority queue. Show that `Heap_Extract_Max` algorithm is essentially a part of Heapsort. Derive and compare the time complexities of Heapsort and `Heap_Extract_Max`. **(17 marks)** [CSE 105 | 2021–22]

Q12. You are given two balanced binary search trees with m and n elements respectively. Write a function that merges them into a balanced BST in $O(m + n)$ time. **(15 marks)** [CSE 203 | 2021–22]

Q13. Calculate for the min heap shown in the given figure. Show the resulting min heap after deleting the minimum element (10), including intermediate steps. **(10 marks)** [CSE 203 | 2021–22]



Q14. While preparing for this exam, you have found an interesting question in a very old book with missing pages. It talks about a Binary Search Tree (BST) that stores integers in the range from 1 to 100 and proposes to search 22 and print the values encountered on the search path through the BST. Then it provides the following sequences of numbers (Sequences 1–3) and asks the following question: "Are these sequences possible search paths through the BST?"

Sequence 1 : 100, 40, 20, 30, 23, 35, 22

Sequence 2 : 100, 35, 15, 32, 20, 22

Sequence 3 : 50, 60, 90, 40, 75

You have to answer the above question. If your answer is 'yes', please show the path (with branches); if 'no', explain why.

[Hint: you were thinking whether there was a missing page with a figure showing the BST, but your genius little brother smiled and hinted that no need to check for a figure as the answer lied already within the sequences provided.] **(9 marks)** [CSE 203 | 2020–21]

Q15. In a Binary Search Tree (BST), the successor of a node N is defined as the node N_{suc} with the smallest key greater than the key of N. Similarly, the predecessor of N is defined as the node N_{pre} with the largest key smaller than the key of N. Now answer the following questions (i)–(iii):

(i) Write an efficient algorithm that will store the keys of the successor and predecessor of each node. Assume that you have two variables, `suc` and `pre`, respectively in the node class/data structure for that purpose. **(7 marks)**

(ii) Discuss an efficient implementation strategy of `removeFullNode(`N`,` N_{suc}`)` (as defined below) with the help of appropriate illustrative examples. Analyze the time complexity of your strategy.

`removeFullNode(`N`,` N_{suc}`)`	N is a full node that has to be removed from the BST. N_{suc} is the successor of N.

(12 marks)

(iii) Your genius little brother suggests that an equivalent strategy is available if we use N_{pre} instead of N_{suc}. Do you agree with him? Justify your answer using the same illustrative example you showed in part 2.b(ii). **(7 marks)**

[CSE 203 | 2020–21]

Q16. Assume that a binary heap is stored in an array called `treeNodes`, which has a capacity of 100 array elements, with array index 0 not being used. If the binary heap currently contains 86 keys, answer the following questions with proper explanation:

(i) Is `treeNodes[44]` a leaf node?

(ii) How many children does `treeNodes[43]` have?

(iii) Is the subtree rooted at `treeNodes[8]` a full binary tree?

(iv) How many levels are there in the subtree rooted at `treeNodes[8]` including `treeNodes[8]`?

(v) Including the root node (`treeNodes[1]`), how many levels are there in the binary heap?

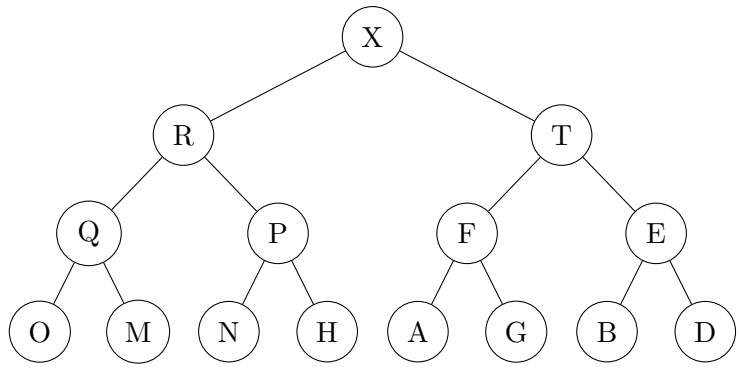
(vi) How many leaf nodes are there?

(12 marks) [CSE 203 | 2020–21]

Q17. Show the steps of building a max-heap by inserting the numbers $45, 36, 27, 14, 86, 19, 69, 10$ one by one. Show the memory representation of the heap. Then write down the steps of sorting these numbers using heap sort. Show the analysis of how `Build-Max-Heap` can be done in $\Theta(n)$ time. **(10+5+10+10 marks)** [CSE 203 | 2019–20]

Q18. Assume you are given a Max Priority Queue of n elements. Write a function that first finds and then deletes the minimum element of the priority queue. **(10 marks)** [CSE 203 | 2018–19]

Q19. Consider the following max-heap of letters:



Draw the results of inserting S into the above max-heap. **(7 marks)** [CSE 203 | 2018–19]

Q20. Suppose you have a set of numbers where you do a constant number of insertions and a linear number of lookups of the maximum over some time period. Compare using a heap vs. a binary search tree for maintaining this set:

(a) What is the asymptotic upper bound on the *average-case* runtime for each, and which structure should you choose?

(b) What is the asymptotic upper bound on the *worst-case* runtime for each, and which structure should you choose?

(12 marks) [CSE 203 | 2018–19]

Q21. Write a function `int treeHeight()` for a Binary Search Tree that computes the height of the tree. Give a worst-case Big-O running time in terms of n. **(8 marks)** [CSE 203 | 2018–19]

Q22. Write a function `void deleteMax()` for a Binary Search Tree that deletes the maximum element (or does nothing if the tree is empty). Give a worst-case Big-O running time in terms of n. Do not assume the tree is balanced. **(7 marks)** [CSE 203 | 2018–19]

Q23. Write a method `isHeap()` that determines whether a particular binary tree is a heap. What is the Big-O bound on the running time of this algorithm? **(8 marks)** [CSE 203 | 2018–19]

Q24. Construct a max-heap with the data $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$ by insertion and adjustment. Show the changes in values at each step. Carry out complexity analysis for both algorithms. **(5+5+10 marks)** [CSE 203 | 2017–18]

Q25. By writing a procedure, explain the three cases that may arise for deleting a node from a binary search tree. **(15 marks)** [CSE 203 | 2016–17, 2015–16]

Q26. Write a function to find the smallest element in a max binary heap. **(8 marks)** [CSE 203 | 2016–17]

Q27. Write the differences between a standard queue and a priority queue. Write two functions that implement the DECREASE-KEY and INCREASE-KEY operations in a max-heap. **(10 marks)** [CSE 203 | 2016–17]

Q28. Write an algorithm that sorts the keys of a binary search tree. Draw the BST that results after inserting the keys $43, 34, 48, 59, 65, 73, 88, 81, 92, 79$ (in that order) into an initially empty BST. Also draw a BST of height three using these keys. **(10 marks)** [CSE 203 | 2016–17]

Q29. Write a function to find the largest element in a min binary heap. Assume the array representing a min binary heap contains the values $2, 8, 3, 10, 16, 7, 18, 13, 15$. Show the contents of the array after deleting the minimum element. **(10 marks)** [CSE 203 | 2015–16]

Q30. Prove that a min binary heap can be built in linear time. **(10 marks)** [CSE 203 | 2015–16]

Q31. Write two procedures for finding the successor element and the predecessor element in a binary search tree. **(10 marks)** [CSE 203 | 2015–16]

Q32. What are the main operations of a priority queue? Explain how one can implement these operations using a heap. **(15 marks)** [CSE 203 | 2014–15, 2011–12]

Q33. Write an algorithm for building a heap. Show that an n-element heap can be built in $O(n)$ time. **(10 marks)** [CSE 203 | 2014–15]

Q34. Draw the binary search tree that results after successively inserting the keys $15, 8, 24,$ $12, 40, 18, 20, 23$ into an initially empty BST. Then, by using necessary rotations, redraw the BST so that it becomes an AVL tree. **(10 marks)** [CSE 203 | 2014–15]

Q35. Suppose we create a binary search tree by inserting the following values in the given order: $50, 10, 13, 45, 55, 110, 5, 31, 64, 47$. Answer the following questions:

(a) Draw the binary search tree.

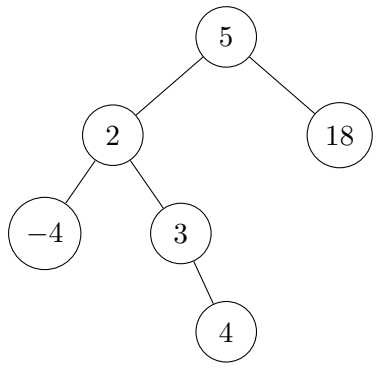
(b) Show the output values if we visit the tree using pre-order traversal.

(c) Show the output values if we visit the tree using post-order traversal.

(d) Show the resulting trees after deleting 47, 110, and 50 (each deletion applied on the original tree).

(5+4+4+6=19 marks) [CSE 203 | 2014–15]

Q36. Give the idea of a data structure that can be used to implement a general tree where nodes can have more than two children. Re-draw the following binary search tree using your proposed data structure:



(8 marks) [CSE 203 | 2014–15]

Q37. Assume an object which has two elements {key, element}. An array of such objects is given. Then:

(a) Convert the array into a Max-heap.

(b) Insert $(36, B)$ in the resultant Max-heap.

(c) Show each step of Heap Sort over the resultant Max-heap.

Array: $[14/U,\ 11/E,\ 13/T,\ 12/C,\ 26/S,\ 19/E,\ 20/W,\ 24/P,\ 18/O,\ 17/L,\ 7/A,\ 8/S,$ $9/H,\ 10/I]$ **(8+8+6=22 marks)** [CSE 203 | 2013–14]

Q38. Write down two algorithms for constructing a max-heap and deduce their complexities. Discuss how Carlsson reduces the leading coefficient in the complexity of the Heapsort algorithm. **(10 marks)** [CSE 207 | 2013–14]

Q39. Write the differences between a standard queue and a priority queue. Write the property of a Max-Heap. Write an algorithm for building a Max-Heap. Analyze the time complexity of the algorithm. **(3+3+7+7=20 marks)** [CSE 203 | 2012–13]

Q40. Sort the array $[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]$ using Heapsort, showing the values at the right of the nodes at each step. **(8 marks)** [CSE 203 | 2011–12]

Q41. What is a priority queue? Write three applications of a priority queue. **(2+3=5 marks)** [CSE 203 | 2011–12]

Q42. Explain the insertion and deletion operations in a binary search tree with illustrative examples. **(14 marks)** [CSE 203 | 2011–12]

BUET · CSE 105

DSA

Graphs & Graph Representations

Adjacency Matrix, Adjacency List, Graph Properties

T7 — Graphs & Graph Representations

Q1. Given a graph $G = (V, E)$ with unique edge weights, let $(S, V - S)$ be a cut in G such that S is non-empty and $S \neq V$. Show that the minimum weighted crossing edge between any such cut in G is included in every minimum spanning tree. **(15 marks)** [CSE 105 | 2023–24]

Q2. Given an adjacency list representation of a directed graph $G = (V, E)$, sketch an efficient algorithm to find its transpose graph $G^T = (V, E^T)$ where $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. Analyze the time complexity of your algorithm. Also evaluate how long it takes to compute the in-degree and out-degree of every vertex of G. **(18 marks)** [CSE 105 | 2022–23]

Q3. A DAG is given to you. Find the maximum number of edges that can be added to this DAG such that the resulting graph is still a DAG (i.e., adding even one more edge would create a cycle). **(10 marks)** [CSE 203 | 2021–22]

Q4. Consider the adjacency matrix representation of a directed graph G. Draw the graph and find its adjacency list representation considering the vertices in ascending order. Find the shortest path distance of every vertex from vertex 1 using BFS. Show the queue status when vertex 10 is just enqueued. Prove that the distance of the most recent vertex (tail in the queue) is at most one larger than the distance of the oldest vertex (head in the queue). **(20 marks)** [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Q5. Write a Graph (directed) data structure using a linked list. Write a delete-node operation for the data structure. **(8+7 marks)** [CSE 203 | 2019–20]

Q6. Suppose you are going to represent a very dense graph and want to optimize both space and time for accessing an edge. Which representation (adjacency list or adjacency matrix) would you prefer? Justify your answer. **(7 marks)** [CSE 203 | 2017–18]

Q7. Compare adjacency matrix and adjacency list representations of a graph. **(10 marks)** [CSE 203 | 2016–17 ,2013–14]

Q8. Define graph, path, cycle, and subgraph with examples. **(10 marks)** [CSE 203 | 2015–16]

Q9. For each of the following operations in an undirected graph, find the worst-case running times in Big-O notation using (i) an adjacency matrix and (ii) an adjacency list:

(a) Determine whether a given vertex is connected to no other vertex.

(b) Determine whether the graph has a vertex that is connected to no other vertex.

(c) Determine whether a given vertex is connected to every other vertex. Give the idea of your algorithm for the adjacency list case.

(4+4+6=14 marks) [CSE 203 | 2014–15]

Q10. Give a brief description of: (i) Graph, (ii) Forest, (iii) Tree, (iv) Spanning subgraph. **(1.5+1.5+1.5+2.5=7 marks)** [CSE 203 | 2013–14]

Q11. Show the adjacency matrix and adjacency list for the following graph. Calculate the number of bytes required in both representations. Assume that the storage requirement of a pointer and an integer is 4 bytes.

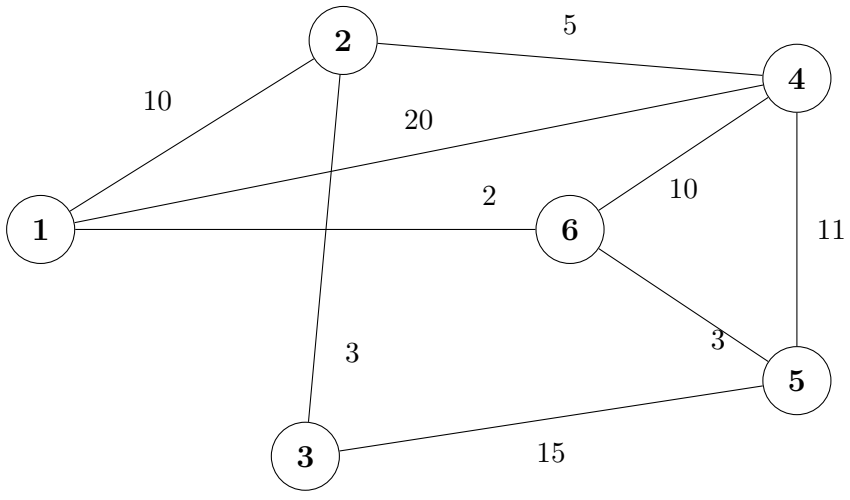
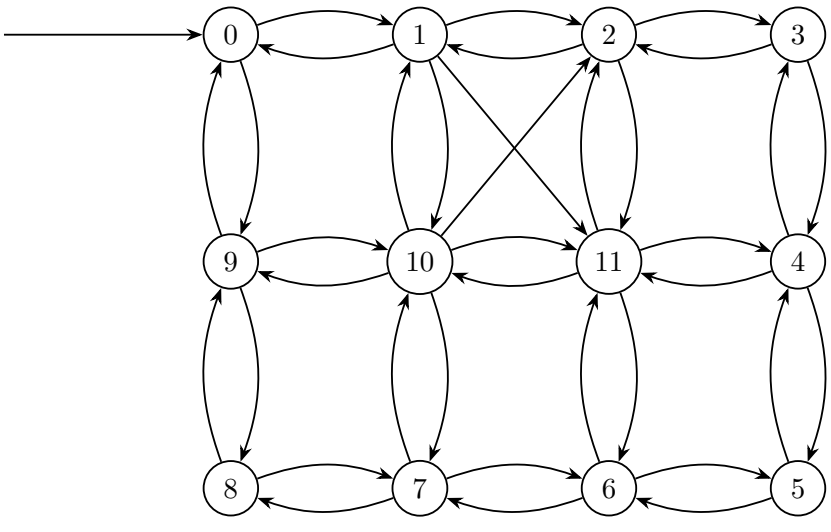
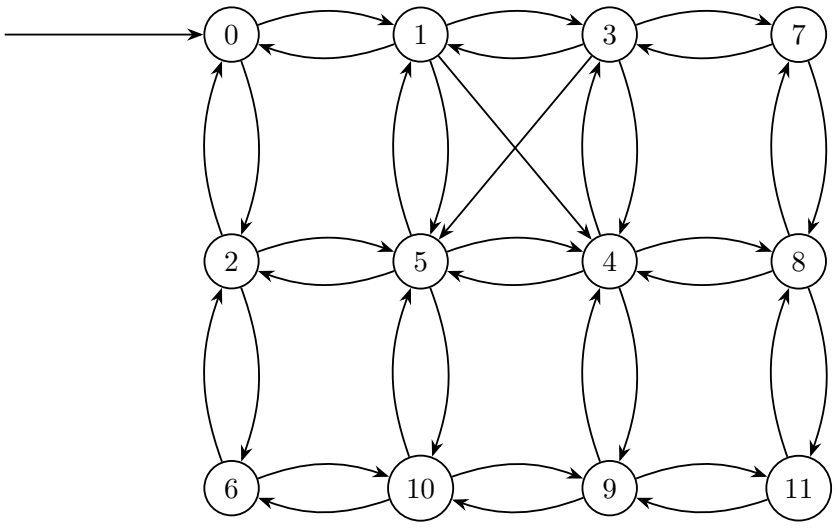


Figure 1: Weighted Graph

Q12. Your friend has employed two different traversal algorithms, *Algo I* and *Algo II*, on a graph and shown the output in Figure Q.5(c)-I and Figure Q.5(c)-II respectively. In both cases, the traversal starts at the node shown by the block right arrow. In the output, nodes are labeled with the order they are traversed. Identify the traversal algorithm for Figure Q.5(c)-I and Figure Q.5(c)-II and justify your answer. **(10 marks)**



FigureQ.5(c) − I



FigureQ.5(c) − II

BUET · CSE 105

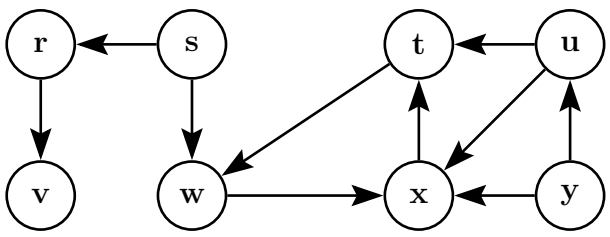
DSA

Graph Traversals: DFS & BFS

DFS Algorithm, BFS Algorithm, Edge Classification

T8 — Graph Traversals: DFS & BFS

Q1. Using a stack, construct an iterative (non-recursive) algorithm for depth first search (DFS) of a graph. Show how the state of the stack changes over time using the graph given in the figure. Analyze the complexity. Modify the algorithm to print the type of each edge it encounters. **(20 marks)** **[CSE 105 | 2023–24]**

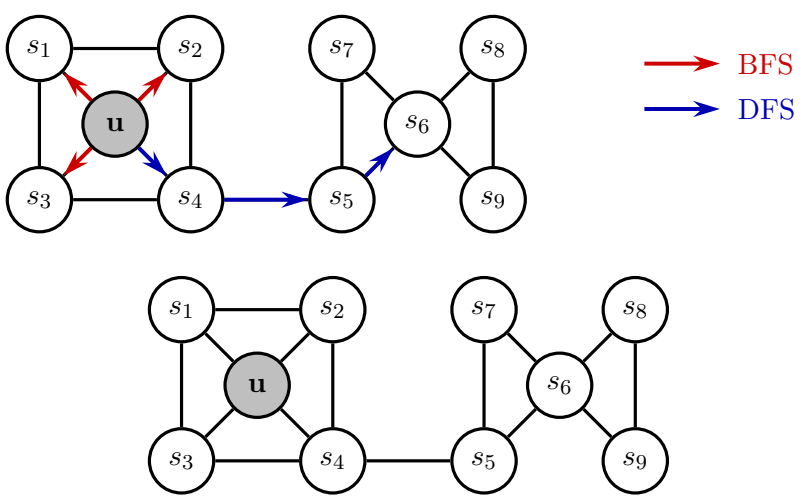


Q2. Define the four types of edges that a DFS on a directed graph yields. Modify the pseudocode of the original DFS algorithm to print all the edges along with their types. Justify whether your algorithm will work correctly. **(18 marks)** **[CSE 105 | 2022–23]**

Q3. Classify the edges of the directed graph G after running DFS on the graph considering the vertices in ascending order. Prove that an edge (u, v) is a tree edge if $u.d < v.d < v.f < u.f$. **(15 marks)** **[CSE 105 | 2021–22]**

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Q4. You are given the following undirected graph to traverse using both BFS and DFS starting from vertex u. Show the simulation steps for labeling the vertices in order of visits. In BFS, the label of a vertex is its level; in DFS, label each vertex with its discovery and finishing times. Also write the pseudocodes for BFS and DFS. **(10+10 marks)** **[CSE 203 | 2019–20]**



Q5. Explain when there is no back edge if DFS is performed on an undirected graph. **(8 marks)** **[CSE 203 | 2018–19]**

Q6. Give the running times of DFS using adjacency lists and adjacency matrix. **(8 marks)** **[CSE 203 | 2018–19]**

Q7. Would you use BFS or DFS to find any path from a start vertex to a goal in a graph with many long paths? Justify. **(10 marks)** **[CSE 203 | 2018–19]**

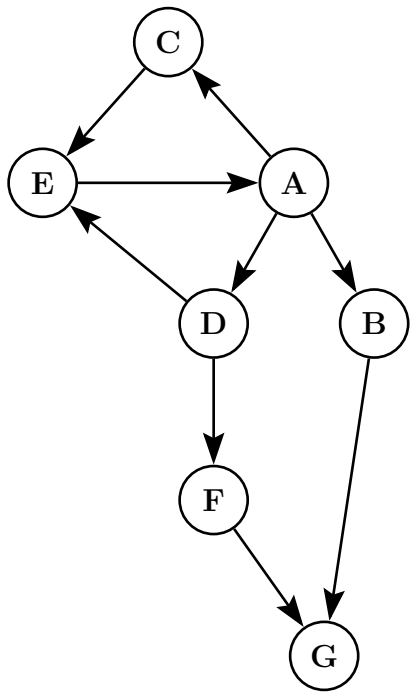
Q8. "Given a directed graph G, both BFS and DFS from a particular starting vertex s will traverse the same set of vertices." Do you agree with this statement? Justify your answer. **(8 marks)** **[CSE 203 | 2017–18]**

Q9. Given an undirected graph $G = (V, E)$, give a linear-time algorithm to check whether there is a cycle of odd length in the graph. Briefly justify its correctness. **(15 marks)** **[CSE 207 | 2015–16]**

Q10. Consider the following directed graph. Answer the following questions:

(a) Perform depth first search on the graph starting from vertex A. Choose the smallest (in alphabetical order) vertex when there is a choice. Find the discovery time and finishing time of each vertex.

(b) Perform breadth first search on the graph starting from vertex A. Choose the smallest (in alphabetical order) vertex when there is a choice. Find the distance and parent of each vertex.



(6+6=12 marks) [CSE 203 | 2014–15]

Q11. Suggest an algorithm based on DFS that counts the number of connected components in an undirected graph. **(8 marks)** [CSE 203 | 2014–15]

Q12. Consider the graph G. Perform the following:

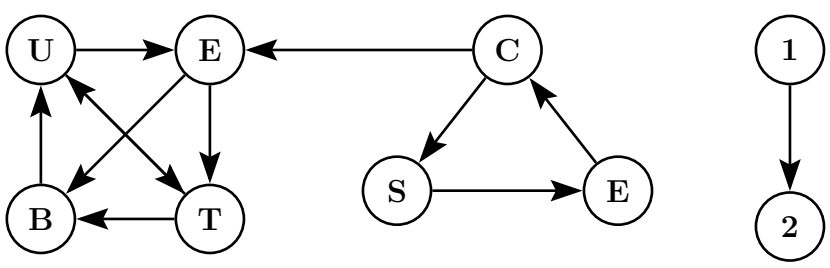
(a) Write the adjacency matrix and adjacency list of graph G.

(b) Draw the BFS forest of graph G assuming B as the starting vertex.

(c) Write the sequence of vertices if you explore graph G starting from vertex B in DFS.

(d) Write the tree edges, back edges, forward edges, and cross edges.

(6+6+4+6=22 marks) [CSE 203 | 2013–14]



Q13. Write an algorithm for DFS traversal. Analyze time complexity. **(8+7=15 marks)** [CSE 203 | 2012–13]

Q14. Consider the graph G. Perform the following:

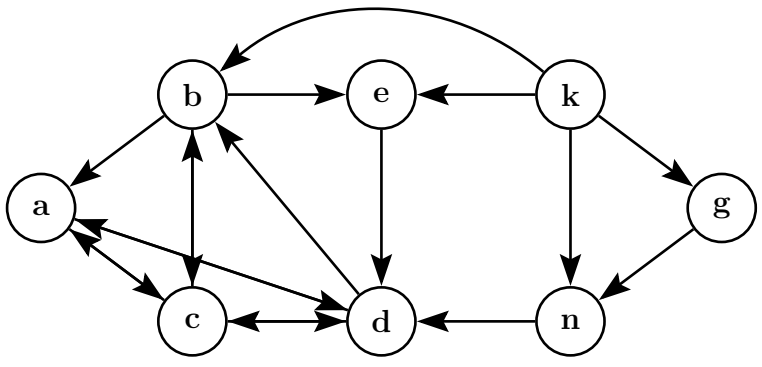
(a) Write the adjacency matrix and adjacency list of the graph G.

(b) Draw the BFS forest of the graph G assuming b as the starting vertex.

(c) Write the sequence of vertices if you explore the graph G starting from vertex b in DFS.

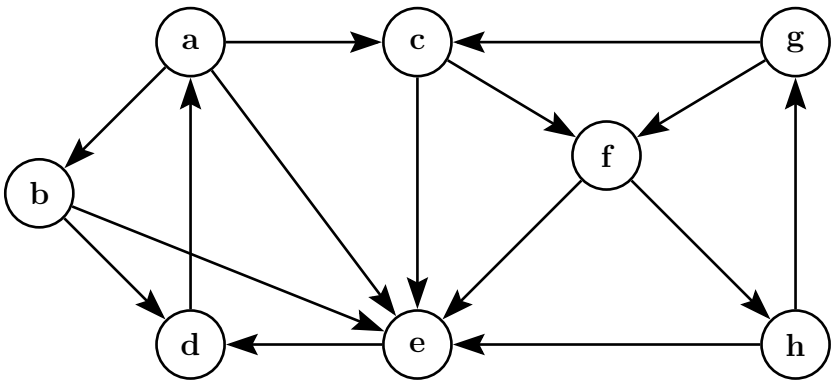
(d) Write the tree edges, back edges, forward edges, and cross edges.

(6+4+4+6=20 marks) [CSE 203 | 2012–13]

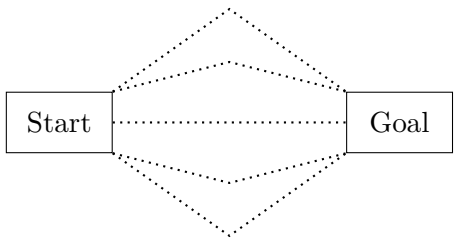


Q15. Write an algorithm for DFS traversal of a given graph. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | 2011–12]

Q16. Write the sequences of the vertices if you explore the given graph G using BFS and DFS. Classify the edges of G for your DFS traversal. **(4+6=10 marks)** [CSE 203 | 2011–12]

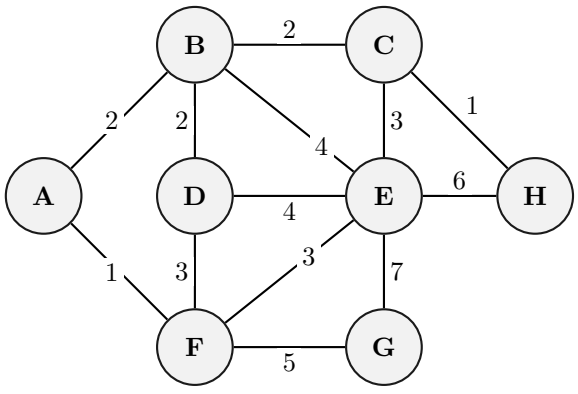


Q17. Suppose you have a graph like the one below. The dots signify some part of the graph that you don't know exactly, not a straight link. You know however, that there are many paths from the start to the goal. You also know that they tend to be rather long. Suppose that you want to implement a program that searches for a path and returns the first one it can find. You have no need for finding the optimal path in any sense, just any path will do. Would you want to use BFS or DFS as the basis for your program? Justify your answer.



(10 marks)

Q18. Consider the weighted graph G as shown in Figure 1. Then draw the adjacency lists of the weighted graph G. Whenever there is a choice of vertices to explore, always pick the one that is alphabetically first.



BUET · CSE 105

DSA

Applications of DFS & BFS

Topological Sort, SCC, Shortest Paths, Dijkstra

T8a — Applications of DFS & BFS

Q1. Explain why the topological sort algorithm cannot run on a directed graph that contains cycle(s). Derive the component graph of the directed graph G represented by the given adjacency matrix. Then find the topological sorting of the vertices of the component graph. Show every step. **(15 marks)** [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Q2. How do you determine whether a given graph is bipartite? **(10 marks)** [CSE 203 | 2019–20]

Q3. Write a function to find the lexicographically smallest topological ordering of a directed graph with N vertices and M edges that may contain cycles. Print -1 if the graph has cycles. **(8 marks)** [CSE 203 | 2018–19]

Q4. The following is Dijkstra's algorithm. Perform a line-by-line time-complexity analysis for an input graph G where an adjacency list is used for graph representation and an ordinary array is used for storing $d[\,]$ values.

```
1:  Q ← V[G]
2:  for each v ∈ Q do
3:      d[v] ← ∞
4:  end for
5:  d[s] ← 0; S ← ∅
6:  while Q ≠ ∅ do
7:      u ← ExtractMin(Q)
8:      S ← S ∪ {u}
9:      for each v ∈ u.Adj[ ] do
10:         if v ∈ Q and d[v] > d[u] + w(u, v) then
11:             d[v] ← d[u] + w(u, v)
12:         end if
13:     end for
14: end while
```

(15 marks) [CSE 207 | 2018–19]

Q5. Johnson's Algorithm uses Bellman-Ford and Dijkstra's algorithms to compute All-Pairs-Shortest-Paths efficiently on sparse graphs. Explain the basic clever idea of Johnson's Algorithm. **(15 marks)** [CSE 207 | 2018–19]

Q6. Briefly explain how a directed acyclic graph $G = (V, E)$ can be topologically sorted so that if (u, v) is an edge then u appears before v in the ordering. Pseudocode is not needed. **(8 marks)** [CSE 203 | 2017–18]

Q7. Suppose you are developing a feature for Facebook. You have access to everyone's friend list and the goal is to show how any two users are connected through a chain of "a friend of a friend". For two users A and B, show a sequence of individuals $X_0, X_1, \ldots, X_n$ where $X_0 = A$, $X_n = B$, and X_i is friends with X_{i+1}, such that the length of the sequence is minimized. Explain how this can be formulated as a graph problem and what algorithm you would use. **(7 marks)** [CSE 203 | 2017–18]

Q8. How can BFS be adapted to solve the single-source shortest path problem in an edge-weighted graph with unequal weights? Explain. **(8 marks)** [CSE 207 | 2017–18]

Q9. Explain the intuitive idea behind the Bellman-Ford algorithm. How can you detect a negative cycle using Bellman-Ford? **(6+2 marks)** [CSE 207 | 2017–18]

Q10. Write Dijkstra's algorithm for the single-source shortest path problem. Prove its correctness. Analyze the time complexity of the algorithm. What would be the running time if a Fibonacci heap is used instead of a binary heap? **(5+5+7+5 marks)** [CSE 207 | 2017–18]

Q11. Design a linear-time algorithm for each of the following. Briefly describe the ideas (pseudocode is not needed):

(a) Given an undirected graph G and a particular edge (u, v) in it, determine whether G has a cycle containing (u, v).

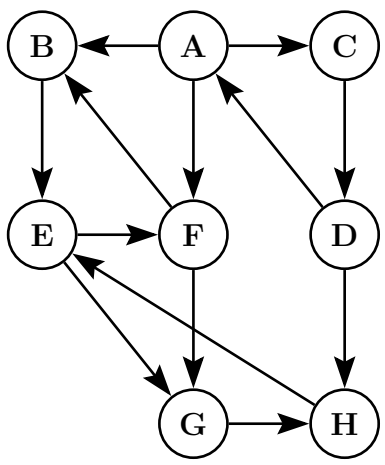
(b) Given an undirected and unweighted graph G, a particular edge (u, v), and two vertices s and t, determine whether there is a shortest path from s to t containing the edge (u, v).

(7+8 marks) [CSE 203 | 2016–17]

Q12. Write the properties satisfied by shortest paths of a graph. Show by example that Dijkstra's algorithm gives wrong results for a graph with negative-weight edges. Write an algorithm that finds shortest paths in a DAG and analyze its time complexity. **(15 marks)** [CSE 207 | 2016–17]

Q13. Prove that the Bellman-Ford algorithm returns TRUE if the input graph contains no negative-weight cycles reachable from the source, and FALSE otherwise. **(10 marks)** [CSE 207 | 2016–17]

Q14. Run DFS on a given graph starting from vertex A and show the discovery and finishing times for each vertex as well as the edge types. Decompose the graph into strongly connected components (SCCs). **(15 marks)** [CSE 203 | 2016–17]



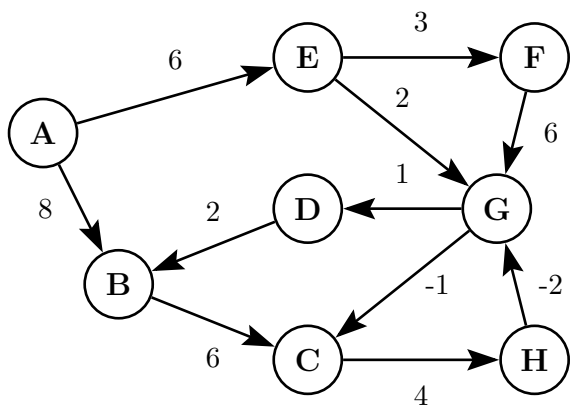
Q15. Suppose a CS curriculum consists of n mandatory courses. The prerequisite directed graph G has a node for each course and an edge from v to w if v is a prerequisite for w. Give a linear-time algorithm that computes the minimum number of semesters necessary to complete the curriculum (a student may take any number of courses per semester). **(15 marks)** [CSE 207 | 2015–16]

Q16. Let C and C' be distinct strongly connected components in a directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ where $u \in C$ and $v \in C'$. Prove that $f(C) > f(C')$, where $f(U)$ denotes the latest finishing time of any vertex in U. **(10 marks)** [CSE 207 | 2014–15]

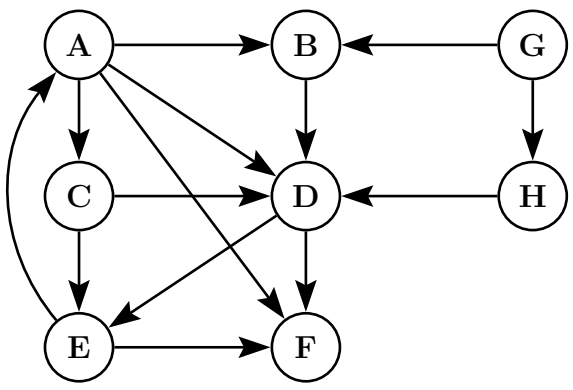
Q17. State and prove the convergence property of edge relaxation in the context of shortest paths. **(8 marks)** [CSE 207 | 2014–15]

Q18. Provide a correctness proof of Dijkstra's algorithm. **(10 marks)** [CSE 207 | 2014–15]

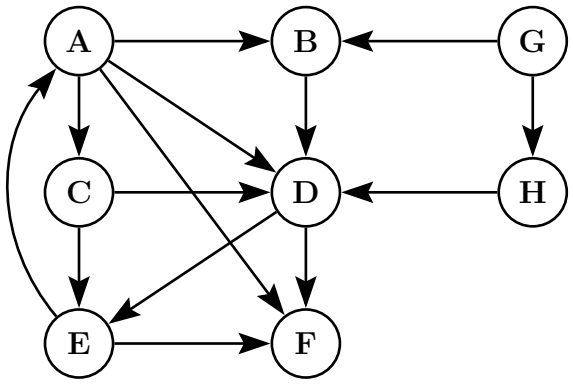
Q19. Apply the Bellman-Ford algorithm to find the shortest paths. Show all relaxation steps. **(10 marks)** [CSE 207 | 2014–15]



Q20. Find the different types of edges if DFS is applied from vertex A. Discuss the edges' properties with respect to the graph. **(6 marks)** [CSE 207 | 2014–15]



Q21. Show the topological sorting order of a given graph with explanation of discovery and finishing times of node exploration. **(6 marks)** [CSE 207 | 2014–15]



Q22. State and prove the convergence property of shortest paths. **(12 marks)** [CSE 207 | 2012–13]

Q23. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. **(20 marks)** [CSE 207 | 2012–13]

Q24. Does Dijkstra's algorithm terminate if it is applied on a graph G that contains a negative-weight cycle? Why or why not? Justify your answer. **(6 marks)** [CSE 207 | 2012–13]

Q25. State the parenthesis theorem for depth-first search. **(6 marks)** [CSE 207 | 2012–13]

Q26. Assume that a breadth-first tree has already been computed with BFS on a graph G with source vertex s. Write an algorithm to print the vertices on a shortest path from s to v. **(8 marks)** [CSE 207 | 2012–13]

Q27. Prove that the component graph of a directed graph is a DAG. **(6 marks)** [CSE 207 | 2012–13]

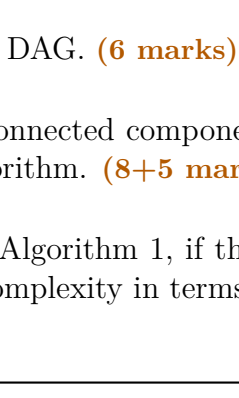
Q28. Write a linear-time algorithm to compute the strongly connected components of a directed graph $G = (V, E)$ using two depth-first searches. Give an example to show every step of the algorithm. **(8+5 marks)** [CSE 207 | 2012–13]

Q29. What does $d[v]$ represent at the end of the execution of Algorithm 1, if the algorithm is applied on an unweighted directed acyclic graph (DAG) G with a source s? What is the runtime complexity in terms of $|V|$ and $|E|$ of Algorithm 1? **(6+6 marks)** [CSE 207 | 2012–13]

Algorithm 2 LongestPath-DAG(G)

```
1:  Compute a topological sorting of G
2:  for each v ∈ V[G] do
3:      d[v] ← 0
4:  end for
5:  for each u in topologically sorted order do
6:      for each v ∈ Adj[u] do
7:          if d[v] < d[u] + 1 then
8:              d[v] ← d[u] + 1
9:          end if
10:     end for
11: end for
```

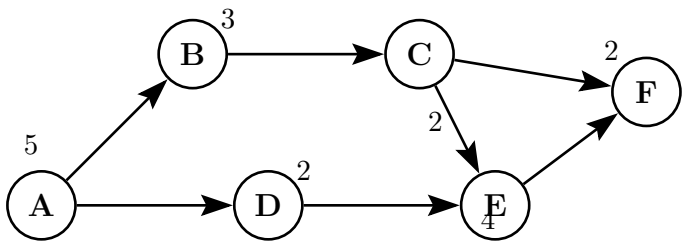
Q30. In Figure 1, the shortest path estimate of each vertex appears within the vertex and the distance to traverse from a vertex to another appears as the weight of the corresponding directed edge. Show the updated shortest path estimates of vertices after relaxing the following edges. **(6 marks)**



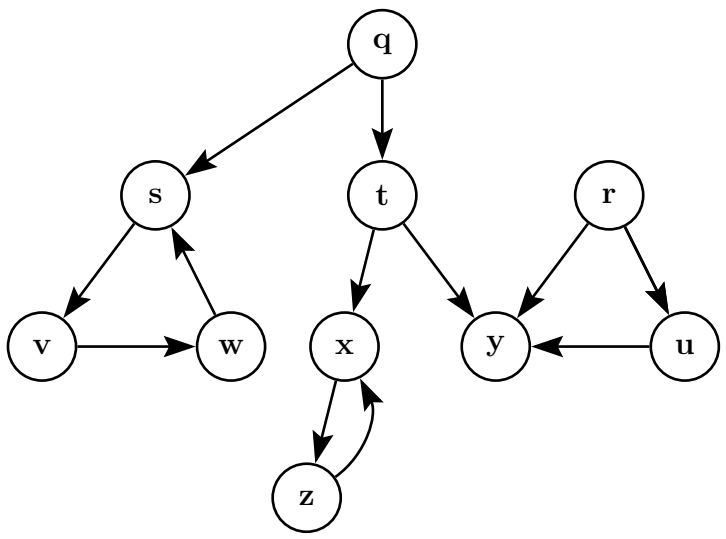
[CSE 207 | 2012–13]

Q31. Prove the correctness of the BFS algorithm. What are "discovery time" and "finish time" in DFS? How can you find the shortest path between two nodes from a BFS tree? **(8+4+6 marks)** [CSE 207 | 2009–10]

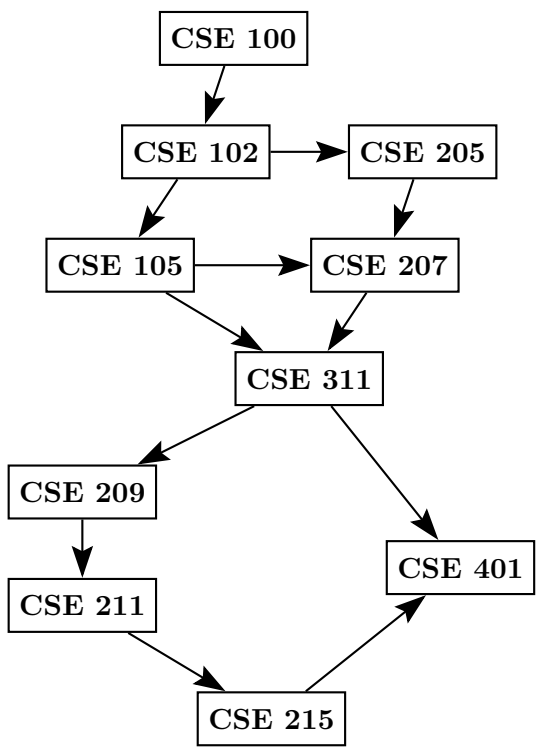
Q32. A project is described by tasks A, B, C, D, E, F taking $5, 3, 2, 2, 4, 2$ days respectively, with precedence constraints (e.g., E cannot start until C and D are completed, but D can be done in parallel with B and C). Write a pseudocode that computes the total time required to complete the project using the PERT technique. **(8 marks)** [CSE 207 | 2009–10]



Q33. Using DFS, compute the strongly connected components and the component graph G^{SCC}. Show the discovery/finish times and the DFS forests created during computation. Start alphabetically. **(17 marks)** **[CSE 207 | 2009–10]**



Q34. Find the topological order of courses for this given prerequisite graph. **(10 marks)** **[CSE 207 | 2006–07]**



BUET · CSE 105

DSA

Divide & Conquer

Mergesort, Closest Pair, Inversions, Strassen

T9 — Divide & Conquer

Q1. Using the recurrence tree method, analyze the runtime complexity of the Mergesort algorithm to sort an array of size n. **(10 marks)** [CSE 105 | 2023–24]

Q2. Explain the run-time complexity of the Mergesort algorithm. First, formulate the run-time as a recurrence relation. Then solve the recurrence relation using both: (i) the recurrence tree method, and (ii) the Master theorem (distinguish the case that is applicable in your analysis). **(2+5+3=10 marks)** [CSE 105 | 2022–23]

Q3. Given a sequence of n numbers $a_1, \ldots, a_n$ (all distinct), define an *inversion* as a pair $i < j$ such that $a_i > a_j$. Present an $O(n \log n)$ divide-and-conquer algorithm to count the number of inversions. Define a *modified-inversion* as a pair where $i < j$ and $a_i > 2a_j$. What modifications are needed in the divide-and-conquer algorithm to count the number of modified-inversions? **(20 marks)** [CSE 105 | 2021–22]

Q4. Given n points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ in the plane, give an efficient divide-and-conquer algorithm to find the pair of points that is closest together. Prove the correctness and analyze the running time of your algorithm. **(17 marks)** [CSE 105 | 2021–22, 2016–17]

Q5. Provide a divide-and-conquer algorithm that takes an n-digit number A as input and efficiently computes the square of A with a faster asymptotic running time than $O(n^2)$. Analyze the running time. **(12 marks)** [CSE 203 | 2021–22]

Q6. Suppose, in a 2-dimensional co-ordinate system, you are given with n points, say $P_1, P_2, \ldots, P_n$, where the x-y co-ordinates of any point P_i is represented as (x_i, y_i). You have been assigned to find the closest pair of points using divide and conquer method.

(i) Write down the approach that you will follow to split these n points into two partitions, say *LEFT* and *RIGHT*, in such a way that each partition contains roughly $\frac{n}{2}$ points.

Now suppose, the *LEFT* and *RIGHT* partitions, respectively calculate d_L and d_R as the distances between the closest pair of points within themselves. Now your task is to check whether there exist a pair of points P_{Li} and P_{Rj} such that $P_{Li} \in LEFT$ and $P_{Rj} \in RIGHT$, and the distance between P_{Li} and P_{Rj} is less than $\min(d_L,\ d_R)$.

(ii) Write down the steps to perform this checking so that the minimum number of comparisons is ensured and the checking can be performed in $O(n)$ time.

(5+10=15 marks) [CSE 203 | 2020–21]

Q7. In a sequence of n distinct numbers $a_1, \ldots, a_n$, a *significant inversion* is a pair $i < j$ such that $a_i > 2a_j$. Give an $O(n \log n)$ algorithm to count the number of significant inversions. Write the corresponding pseudocode. **(10 marks)** [CSE 203 | 2020–21]

Q8. Write an algorithm to find a peak element of an $n \times n$ matrix in $O(n)$ time using divide and conquer. A peak element is one whose left, right, top, and bottom neighbors (if available) are all smaller. Define the recurrence relation and derive the runtime. **(10+5 marks)** [CSE 203 | 2019–20]

Q9. Propose the pseudocode of a merge sort algorithm where the array is divided into three equal parts at each step. Derive the asymptotic runtime of this algorithm. Compare it with standard merge sort (two equal parts) and mention cases when one outperforms the other. **(10+8+7 marks)** [CSE 203 | 2019–20]

Q10. Explain how a recursion tree can help with the substitution method of runtime analysis. **(5 marks)** [CSE 203 | 2019–20]

Q11. Write a divide-and-conquer algorithm that finds the maximum and the minimum of an array of n distinct elements. Write the recurrence relation for the running time $T(n)$ and solve it using the Recursion Tree Method. **(10 marks)** [CSE 203 | 2018–19]

Q12. Write the Mergesort algorithm. Simulate it on the array $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$. **(15 marks)** [CSE 203 | 2017–18]

Q13. Given a sorted array of distinct integers $A[1 \ldots n]$, give a divide-and-conquer algorithm that runs in $O(\log n)$ time to find whether there is an index i for which $A[i] = i$. Write the pseudocode and prove the runtime. **(12 marks)** [CSE 207 | 2014–15]

Q14. Explain the divide-and-conquer technique. What is the function of the pivot element in the Quicksort algorithm? **(3+2=5 marks)** [CSE 203 | 2012–13]

Q15. Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = T(n/4) + T(n/2) + n^2$. **(10 marks)** [CSE 207 | 2012–13]

Q16. What is the divide-and-conquer technique? **(5 marks)** [CSE 203 | 2011–12]

Q17. Write the Mergesort algorithm. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | 2011–12]

Q18. Write the recurrence relation of the Mergesort algorithm. Draw the recursion tree for the recurrence relation of the Mergesort algorithm. **(7 marks)** [CSE 203 | 2011–12]

Q19. In the closest-pair-of-points problem using the divide-and-conquer approach, if q is a point in the left half and r is a point in the

right half, and $d(q, r) < \delta$, where δ is the minimum of distances in each half, then show that each of q and r lies within a distance δ of the line dividing the two halves. Hence show that q and r are within 11 positions of each other in a list sorted by y-coordinate. **(15 marks)** **[CSE 207 | 2011–12]**

BUET · CSE 105

DSA

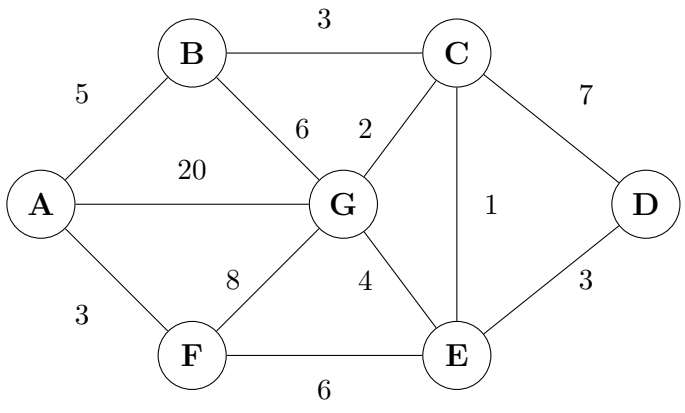
Greedy Methods

MST (Kruskal, Prim), Huffman, Activity Selection, Dijkstra

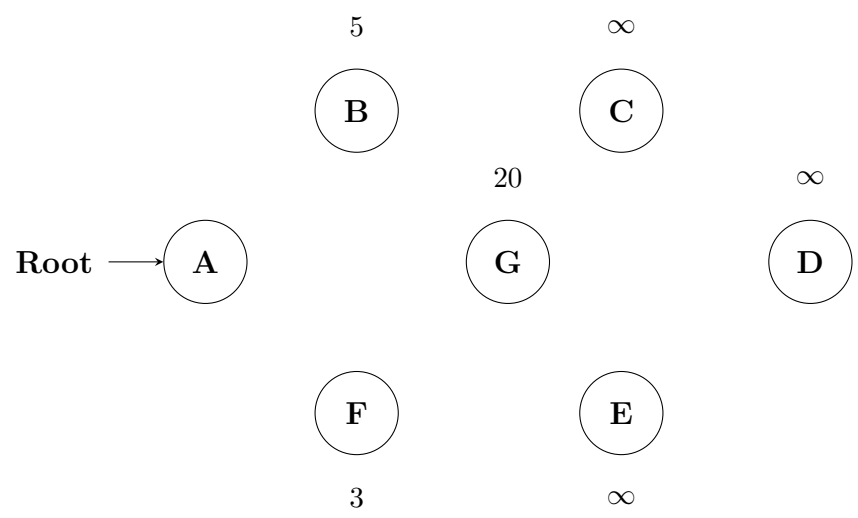
T10 — Greedy Methods

Q1. Define the Activity Selection Problem. Prove that the greedy choice of selecting activities based on earliest finish time yields an optimal solution. **(5+10=15 marks)** [CSE 105 | 2023–24]

Q2. Apply Prim's algorithm to find the minimum spanning tree of the given graph.

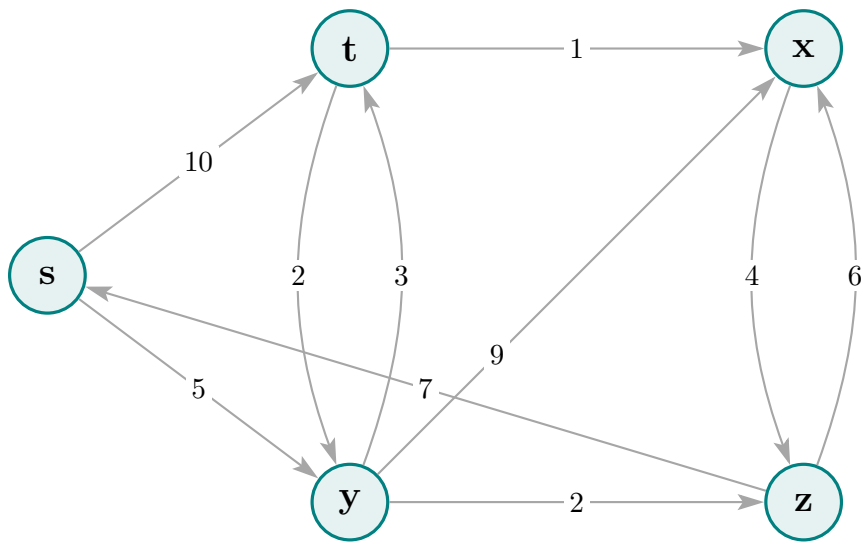


Start with node A as the root node. At each step, show the intermediate MST (i.e., the partial tree) and show the estimated cost to connect each non-visited node (i.e., the minimum weight of an edge connecting an unvisited node to any visited nodes).The first step , where only the root node is included in the tree , is done for you . **(10 marks)** [CSE 105 | 2023–24]



Q3. Argue in favor of the correctness of Prim's algorithm to find a minimum spanning tree of a graph $G = (V, E)$. Prove any non-trivial graph properties used in the argument. **(5+10=15 marks)** [CSE 105 | 2022–23]

Q4. Find the shortest path distance from every node x to node t in the given graph using Dijkstra's Algorithm. Re-draw the diagram for each intermediate step of the algorithm (after relaxation) while annotating each node with the appropriate shortest path distance estimate of that step. **(10 marks)** [CSE 105 | 2022–23]



Q5. Given a set $U = \{x_1, x_2, \ldots, x_n\}$ of n integers and an integer k $(k < n)$, give an optimal greedy algorithm to find a subset S of size k in U such that the summation of the numbers in S is maximum (over all subsets of size k in U). Prove the correctness of your algorithm and analyze its running time. **(15 marks)** [CSE 105 | 2021–22]

Q6. Consider a set of n intervals where each interval i specifies a start time s_i and a finish time f_i $(s_i < f_i)$. Two intervals are compatible if they do not overlap. Give a greedy algorithm to select a compatible subset of maximum possible size. Prove the correctness and analyze the running time. **(16 marks)** [CSE 203 | 2021–22]

Q7. Given a set $S = \{x_1, x_2, \ldots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \cdots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contain all numbers in S. Prove correctness and analyze running time. **(15 marks)** [CSE 203 | 2021–22]

Q8. Once upon a time there was a city that had no proper roads. Getting around the city was particularly difficult after rainstorms because the ground used to become very muddy, cars got stuck in the mud, and people got their boots dirty. The mayor of the city decided that some of the streets must be paved, but he did not want to spend more money than necessary because the city also wanted to build a swimming pool. The mayor therefore specified two conditions:

(i) Enough streets must be paved so that it is possible for everyone to travel from their house to anyone else's house only along the paved roads, and

(ii) The paving should cost as little as possible.

The following figure shows the layout of the city. The number of paving stones in between two houses represents the cost of paving that route. Your task is to apply a greedy based algorithm that you have learned under this course to fulfill the requirement of the Mayor. Note that the bridge in the figure should not be considered as a paving stone. In your answer script, you must draw a simple graph representing the given layout of the muddy city. After that, you have to show the steps of applying your chosen greedy algorithm to find the connected subgraph which contains all the houses of the city (considering them as vertices of your drawn graph) and a subset of roads (considering them as edges) in a way that the total number of paving stones required is the minimum. You have to ensure that your connected subgraph does not contain any cycle, and you need to mention the algorithm that you are adopting to find your desired solution. **(15 marks)** **[CSE 203 | 2020–21]**

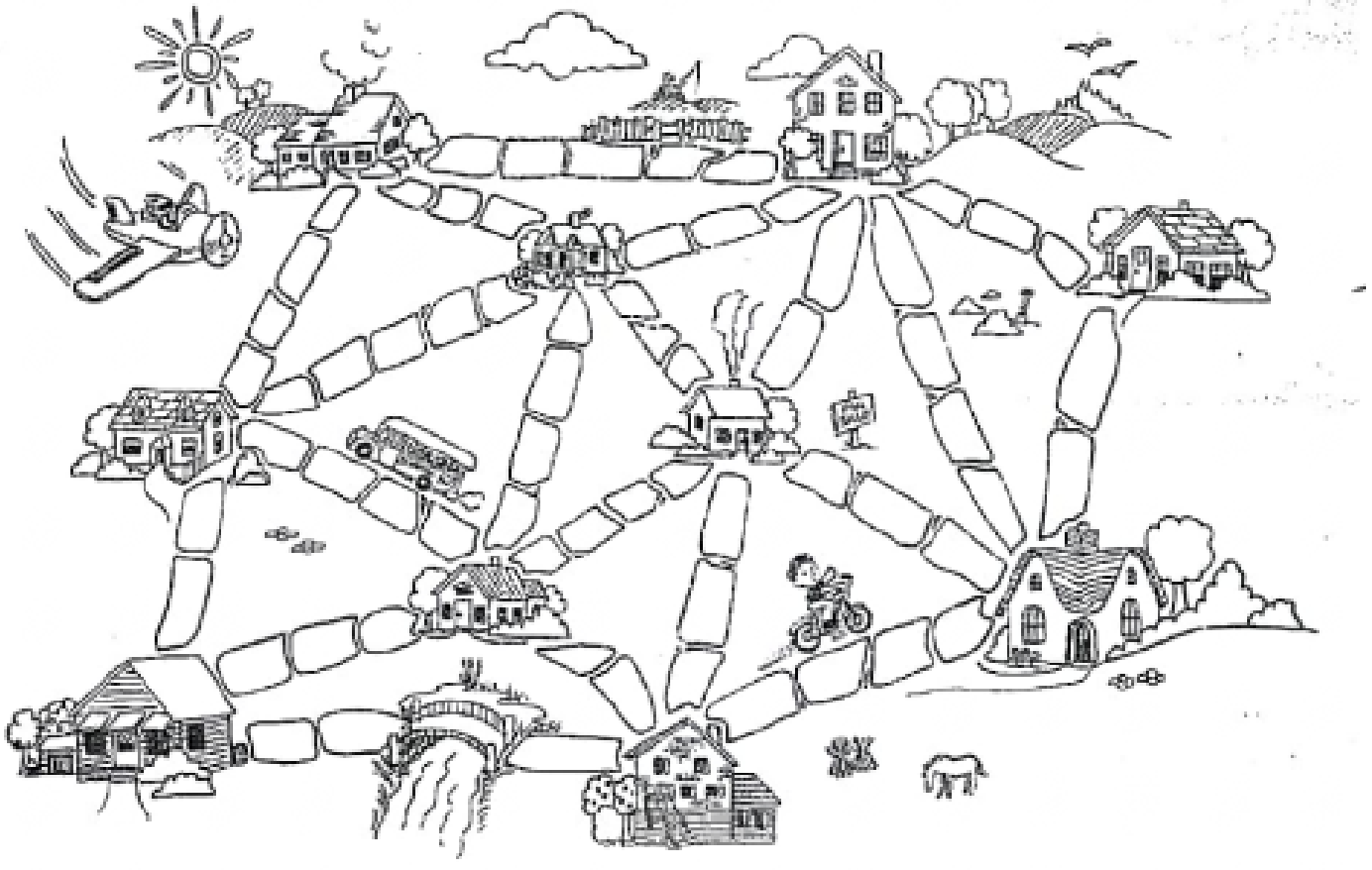


Figure for Ques No. 5(b) – The layout of the muddy city and the paving requirement

Q9. Prove that an optimal solution is always guaranteed if we design an activity-selection greedy algorithm by always choosing the activity with the earliest finish time from the remaining compatible activities. **(10 marks)** **[CSE 203 | 2020–21]**

Q10. Analyze the runtime complexity of MST-Prim's algorithm (as given using a Binary Min-Heap to maintain a minimum priority queue), mentioning the individual runtime of each significant operation. How is this running time improved if a Fibonacci Heap is used instead? **(10 marks)** **[CSE 203 | 2020–21]**

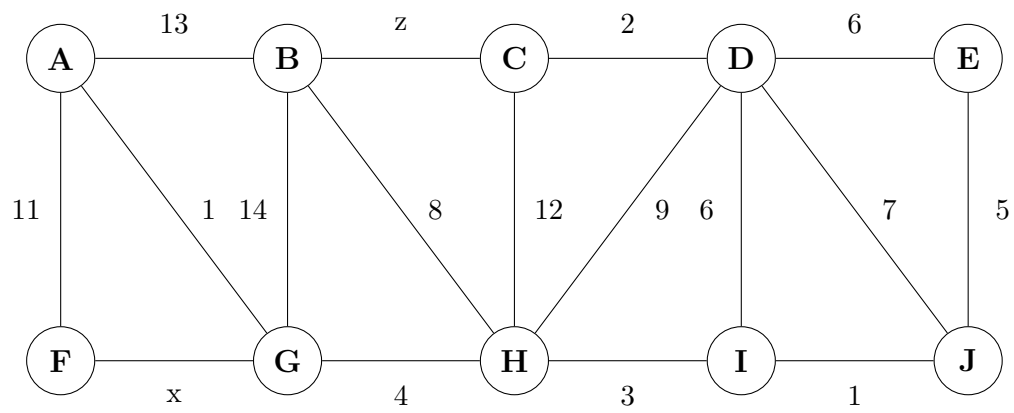
Q11. Write an algorithm to calculate the maximum profit earned by executing tasks within specified deadlines. A task takes one unit of time to execute and cannot execute beyond its deadline; only one task executes at a time. Apply the algorithm to the following data:

Task	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Deadline	7	3	2	6	8	5	6	3	2	4
Profit	9	14	5	10	3	15	5	7	8	6

(10+10 marks) **[CSE 203 | 2019–20]**

Q12. Show that the fractional knapsack problem has the optimal substructure property. Also show that the 0-1 knapsack problem has the overlapping subproblems property. **(10 marks)** **[CSE 203 | 2018–19]**

Q13. Prove that an optimal minimum spanning tree is composed of optimal minimum spanning subtrees. Draw a minimum spanning tree containing the edges with weights x and z. **(15 marks)** **[CSE 207 | 2018–19]**



Q14. Write down an algorithm for job sequencing with deadlines. Solve the problem instance with profit and deadline values (p_i, d_i) as follows: $(7,5),(6,5),(5,2),(4,7),(3,7)$,
$(3,7),(3,6),(2,7),(2,1),(2,5),(2,3),(2,1),(1,1)$. **(10+10 marks)** [CSE 203 | 2017–18]

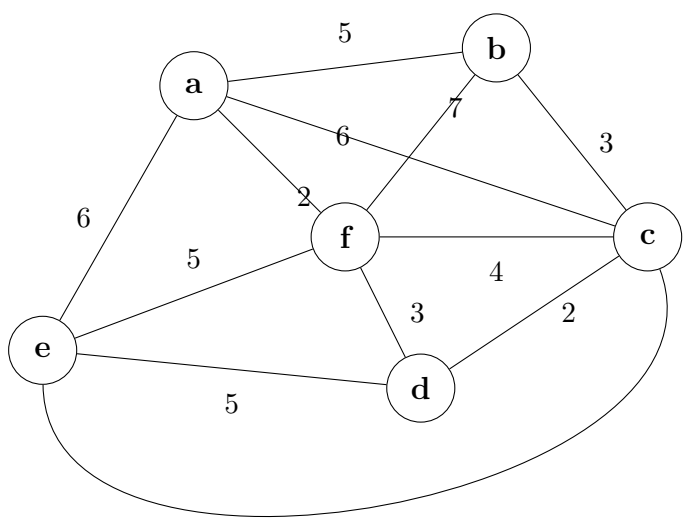
Q15. Give an optimal greedy algorithm for solving the fractional knapsack problem. Prove that your algorithm is optimal. **(13 marks)** [CSE 203 | 2017–18]

Q16. Given a set $S = \{x_1, x_2, \ldots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \ldots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contain all the numbers in S. For example, an interval $[1.5,\ 2.5]$ is a unit length interval (since $2.5 - 1.5 = 1$), and it contains all the real numbers from 1.5 to 2.5 (including 1.5 and 2.5). You have to find the smallest set of such unit length intervals which contains all the real numbers in S. Prove the correctness of your algorithm, and analyze the running time. **(15 marks)** [CSE 203 | 2017–18]

Q17. Write down an algorithm for constructing a minimum spanning tree. Construct the MST for the graph defined by the edges: $ra(7), rx(2), ax(1), xy(5), ya(3), xc(6), cb(4)$,
$bx(2), yb(2)$, where the weight of each edge is given in parentheses. **(10+10 marks)** [CSE 203 | 2017–18]

Q18. What is a spanning tree of a graph? Write Kruskal's algorithm for finding a minimum spanning tree of an edge-weighted graph. Analyze the time complexity of the algorithm by suggesting appropriate data structures. **(3+5+7 marks)** [CSE 207 | 2017–18]

Q19. Find a minimum spanning tree using Prim's algorithm, showing every step. **(10 marks)** [CSE 207 | 2017–18]



Q20. Let G be a weighted connected graph and let V_1, V_2 be a partition of its vertices into two disjoint non-empty sets. Let e be an edge in G with minimum weight among those with one endpoint in V_1 and the other in V_2. Show that there is a minimum spanning tree T of G containing e. **(10 marks)** [CSE 207 | 2017–18]

Q21. Huffman's algorithm is used to obtain an encoding of alphabet $\{a, b, c\}$ with frequencies f_a, f_b, and f_c. In each of the following cases, either give an example of frequencies that would yield the specified code, or explain why the code cannot possibly be obtained:

(a) Code: $\{0, 10, 11\}$

(b) Code: $\{0, 1, 00\}$

(c) Code: $\{10, 01, 00\}$

Then construct Huffman codes for the characters with the following frequencies: A:24, B:12, C:10, D:8, E:9, F:37, using the greedy algorithm. **(6+9 marks)** [CSE 203 | 2016–17]

Q22. Suppose you are given n white and n black dots, lying on a line. The dots are equally spaced but they may appear in any order of black and white (black and white may be interleaved). Design a greedy algorithm to connect each black dot with a (different) white dot so that the total length of wires used to connect the dots is minimal. The length of wire used to connect two dots is equal to their distance along the line. Prove that your algorithm is optimal.

●	●	○	●	○	○	○	●
1	2	3	4	5	6	7	8

For example, an optimal solution to solve the above example is $(1, 3)$, $(2, 5)$, $(4, 6)$, $(7, 8)$ with total wire length $= 2 + 3 + 2 + 1 = 8$. **(15 marks)** [CSE 203 | 2016–17]

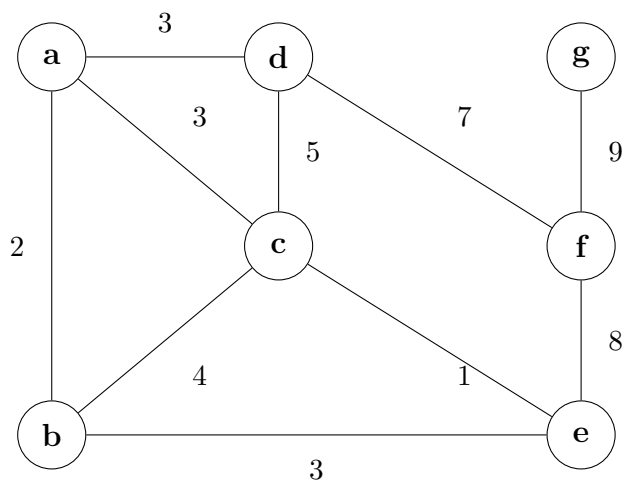
Q23. Write Prim's algorithm that computes a minimum spanning tree of a graph. Analyze its time complexity. **(15 marks)** [CSE 207 | 2016–17]

Q24. Write the correctness proof of Kruskal's algorithm for finding a minimum spanning tree. **(10 marks)** [CSE 207 | 2016–17]

Q25. Compare the key properties of problems that can be solved with greedy method, divide-and-conquer method, and dynamic programming method. **(10 marks)** [CSE 207 | 2016–17]

Q26. There is a network of roads $G = (V, E)$ connecting a set of cities V. Each road in E has an associated length l_e. There is a proposal to add one new road to this network, and there is a list E' of pairs of cities between which the new road can be built. Each such potential road $e' \in E'$ has an associated length. As a designer for the public works department you are asked to determine the road $e' \in E'$ whose addition to existing network G would result in the maximum decrease in the driving distance between two fixed cities s and t in the network. Give an algorithm for solving this problem. Your algorithm should run in $O((V + E + E') \log V)$ time. **(15 marks)** [CSE 207 | 2015–16]

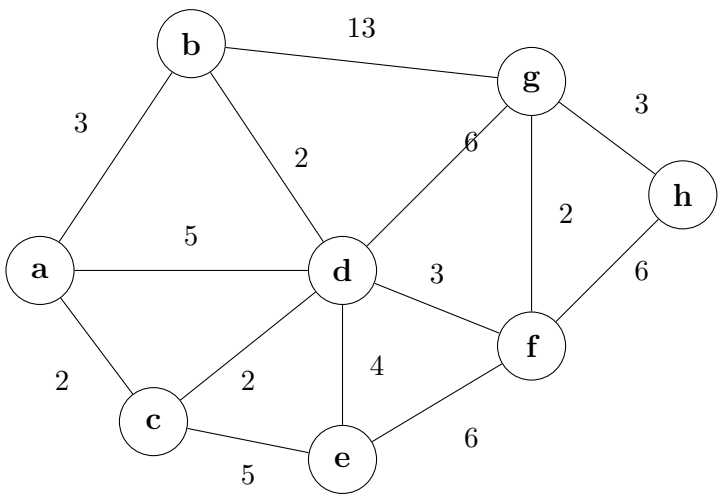
Q27. Find a minimum spanning tree by running (i) Kruskal's algorithm and (ii) Prim's algorithm. Show the order of edges selected in each case (use alphabetical ordering when there is a tie). **(15 marks)** [CSE 207 | 2015–16]



Q28. Under a Huffman encoding of n symbols with frequencies $f_1, f_2, \ldots, f_n$, what is the longest a codeword could possibly be? Give an example set of frequencies that would produce this case. Then construct Huffman codes for the following frequencies: A:45, B:13, C:12, D:16, E:5, F:9, using the greedy algorithm. **(7+8=15 marks)** [CSE 207 | 2015–16]

Q29. Let $G = (V, E)$ be a connected undirected graph with weight function w on E. Let $A \subseteq E$ be included in some MST for G, let $(S, V - S)$ be any cut that respects A, and let (u, v) be a light edge crossing $(S, V - S)$. Prove that (u, v) is safe for A. **(10 marks)** [CSE 207 | 2014–15]

Q30. Find the light and respectful crossing edges for each step of Kruskal's method for building the MST. **(12 marks)** [CSE 207 | 2014–15]

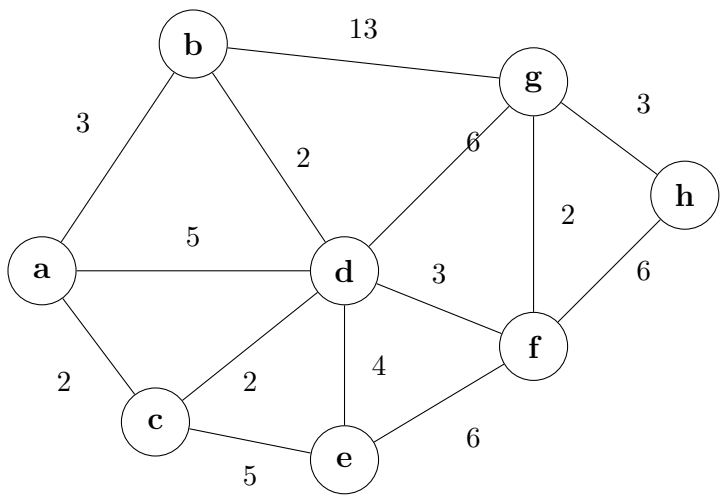


Q31. What is the cost of finding and joining (union) of sets in Kruskal's method? **(8 marks)** [CSE 207 | 2014–15]

Q32. For the graph below, show the value of u, $v.key$, Q for the following algorithm (show each step): **(10 marks)** [CSE 207 | 2014–15]

Algorithm 3 MST(G, w, r)

```
1: Q ← V[G]
2: for each u ∈ Q do
3:     u.key ← ∞
4: end for
5: r.key ← 0;   p[r] ← nil
6: while Q ≠ ∅ do
7:     u ← EXTRACT-MIN(Q)
8:     for each v ∈ Adj[u] do
9:         if v ∈ Q and w(u, v) < v.key then
10:            p[v] ← u
11:            v.key ← w(u, v)
12:        end if
13:    end for
14: end while
```



Q33. *GreedyCell*, a mobile phone operator, wants to place cell towers (base stations) along a long straight country road, to provide network coverage to n houses sparsely scattered along the road. The distance of these houses from the start of the road are, in miles and in increasing order, $d_1, d_2, \ldots, d_n$. Each cell tower has coverage of R miles, i.e., if a house is within R miles of a tower, it would get service from that tower. Now give an efficient algorithm for finding a placement of cell towers that uses as few cell towers as possible to provide coverage to all the n houses.**(12 marks)** [CSE 207 | 2014–15]

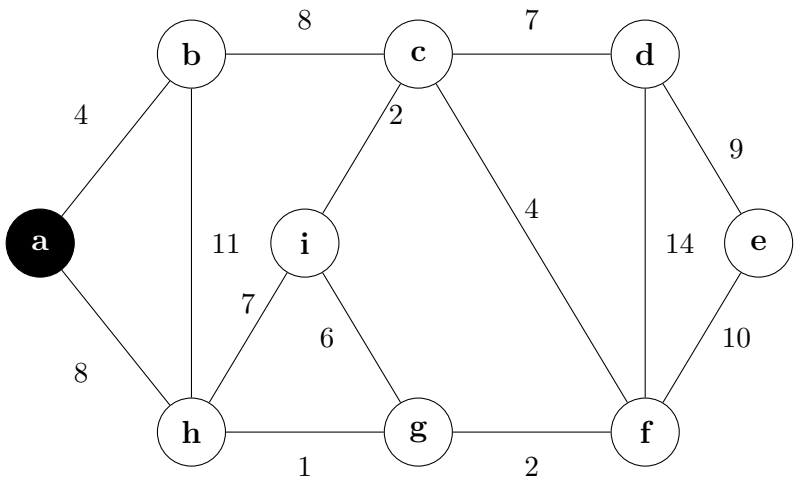
Q34. Find a minimum spanning tree using both Kruskal's algorithm and Prim's algorithm (starting from A) for the graph with edges:
$AB(5), BC(2), AC(4), AD(6), CD(3)$,
$DB(1), DE(6), EB(3), EF(3), FB(4)$. **(15 marks)** [CSE 207 | 2013–14]

Q35. Given a set of jobs with start and finish times, write down an algorithm so that the maximum number of jobs can be processed. Show counterexamples for some natural greedy approaches that will not find optimal solutions. **(10 marks)** [CSE 207 | 2013–14]

Q36. Given the message `abaacbdbebfbedcect', construct a dynamic Huffman tree showing how the tree is updated after sending each symbol and the code of each symbol. **(15 marks)** [CSE 207 | 2013–14]

Q37. Suppose you are given n tasks with start times $S[1 \ldots n]$ and finish times $F[1 \ldots n]$. Schedule as many tasks as possible so that no two overlap. (i) Describe a greedy algorithm that finds the maximum number of non-overlapping tasks. (ii) Prove the correctness of your greedy algorithm. **(15 marks)** [CSE 207 | 2012–13]

Q38. If Prim's algorithm is applied on a given graph, which will be the fifth edge added to the spanning tree? Start from vertex a. **(10 marks)** [CSE 207 | 2012–13]



Q39. "Search spaces for natural combinatorial problems tend to grow exponentially with the size of the input" — explain and justify with necessary examples. **(15 marks)** [CSE 207 | 2011–12]

Q40. Explain why one can solve the fractional knapsack problem by a greedy strategy, but one cannot solve the 0-1 knapsack problem by such a strategy. **(10 marks)** [CSE 207 | 2011–12]

Q41. Prove that the greedy method stays ahead in the interval scheduling problem, and hence show that the greedy method returns an optimal set of jobs. **(15 marks)** [CSE 207 | 2011–12]

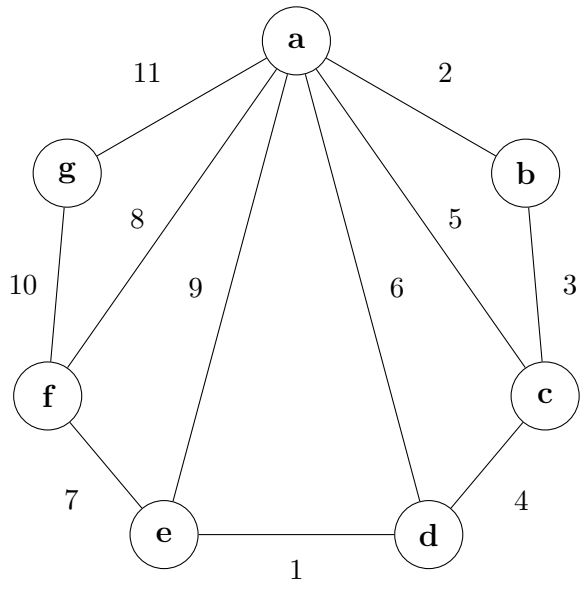
Q42. Prove that if we use a greedy algorithm for the interval partitioning problem, every interval will be assigned a label, and no two overlapping intervals will receive the same label. **(15 marks)** [CSE 207 | 2011–12]

Q43. Explain how Dijkstra's algorithm can be implemented using a priority queue. Find the running time for this algorithm. **(15 marks)** [CSE 207 | 2011–12]

Q44. State and prove the "cut property" and the "cycle property". Explain with necessary examples how these properties are useful in solving the minimum spanning tree problem. **(15 marks)** [CSE 207 | 2011–12]

Q45. Describe, with necessary elaboration on the required operations and data structure, how the union-find data structure can be used for Kruskal's algorithm. **(15 marks)** [CSE 207 | 2011–12]

Q46. What is the criterion for selecting an edge at each step of Kruskal's algorithm for computing MST? How does this criterion guarantee the algorithm gives an MST? Show the steps of Kruskal's algorithm on a given graph. **(17 marks)** [CSE 207 | 2009–10]



Q47. Construct the Huffman tree and show the Huffman coding for the following characters with given frequencies:

Character	a	b	c	d	e	f
Frequency	50	30	20	15	10	5

(10 marks) [CSE 207 | 2009–10]

Q48. Construct a minimum spanning tree using Kruskal's algorithm for the following weighted graph: $AB(2), AC(3), AE(5), BC(1), BD(6), BF(9), CF(3$ $DH(5), FG(4)$. Show every step, giving reasons for discarding any edge. **(15+5=20 marks)** [CSE 207 | 2008–09]

Q49. Discuss how Prim's algorithm constructs a minimum spanning tree. Argue how sets and the disjoint-set union algorithm can be used to detect cycles and improve upon Kruskal's algorithm. **(15 marks)** [CSE 207 | 2008–09]

Q50. Discuss how a Huffman tree can be constructed given a sequence of frequencies corresponding to symbols. Construct a dynamic Huffman tree based upon the message `ABBBCAAAACBBBB`. **(20 marks)** [CSE 207 | 2008–09]

Q51. Suppose you have a washing machine that a number of users want to use. Each user i sends a request for the time interval (s_i, f_i) (start and finish times). The machine can serve only one user at a time, and each user pays 1000/- regardless of duration. Write a greedy algorithm to maximize revenue. Prove the correctness of your algorithm and deduce its running time. **(8+10+5 marks)** [CSE 207 | 2007–08]

Q52. Define the interval coloring problem and write an algorithm to solve it. **(4+8 marks)** [CSE 207 | 2007–08]

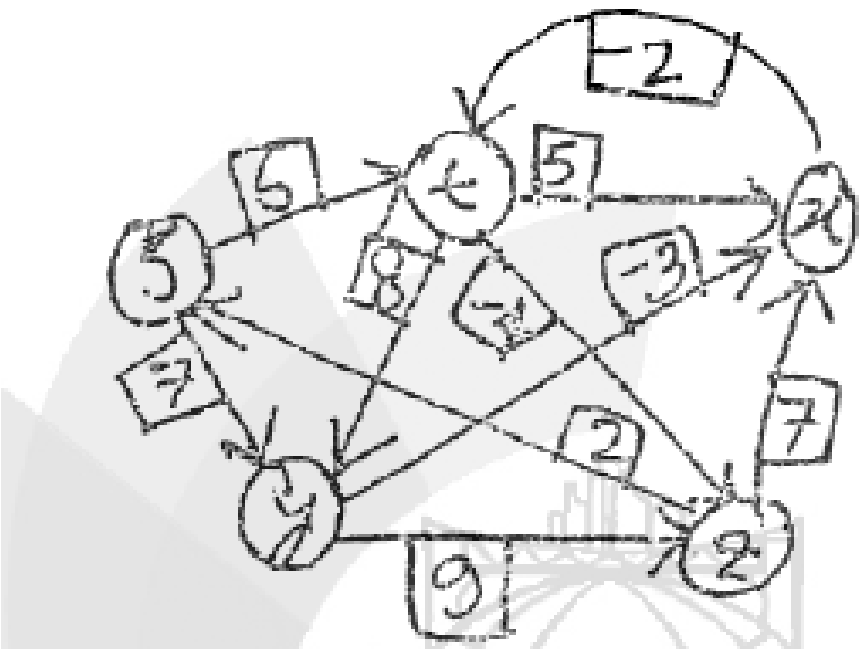
Q53. Describe the main idea of Prim's algorithm to compute a minimum spanning tree. Discuss how Prim's algorithm can be implemented in $O(m \log n)$ time. Discuss the reverse-delete algorithm to compute a minimum spanning tree and prove its correctness. **(6+10+17 marks)** [CSE 207 | 2007–08]

Q54. Write down an algorithm for constructing a minimum spanning tree using Kruskal's algorithm. Show how sets and the disjoint-set algorithm can be used to reduce its complexity. Construct a minimum spanning tree for the graph with edges: $AB(7), AC(3), DA(4), EA(1), BC(91), C$ $FD(3)$. **(5+5+5 marks)** [CSE 207 | 2006–07]

Q55. Construct a Huffman tree for the phrase with symbols and frequencies: A-7, B-4, C-5, D-2, E-20, F-3, G-6, H-8, I-1. **(10 marks)** [CSE 207 | 2006–07]

Q56. There is a set of jobs where each job has an integer deadline and profit, and only one machine is available. To complete a job requires one unit of time; profit is earned only if the job is completed by its deadline. Jobs A, B, C, D, E have deadlines $3, 3, 1, 2, 2$ and profits $1, 5, 10, 15, 20$ respectively. Show the steps of selecting jobs using a greedy strategy. **(8 marks)** [CSE 207 | 2006–07]

Q57. Find the shortest paths starting from vertex s in this given graph. Show the steps of edge relaxation. **(10 marks)** [CSE 207 | 2006–07]



Q58. Once an amateur thief entered into a museum full of tantalizing jewelry, geodes and rare gems. As he is new in this field, he brought just a single backpack with him that has a capacity of 6 kg. The thief planned to get away with the most valuable objects without overloading the backpack. He found out that in a less-secured compartment, there were four (4) solid items, and their information are given in the following table.

Item	Weight (kg)	Value (USD)
Brooch of the queen	3	15
Crown of the king	2	20
Decorated vase	10	30
An ancient coin	2	14

Table for Ques No. 5(a) — Information of valuable items found in the museum

Since the items are unbreakable, for each item, the thief must pick the whole of it or not consider it at all. Now although our thief is amateur in this business, he possesses some algorithmic knowledge. Therefore, he applied a greedy technique and selected the items that gave him the maximum possible profit per weight, and then he left the museum with so much zeal! Now, using the information of the backpack of the thief and the item table given for this question, your first task is to show the steps of finding the maximum possible profit and the corresponding list of items using the technique that the thief adopted. You have also identified that the scenario at the museum was a special one and only dynamic programming is able to find the optimal solution in that case. Therefore, your second task is to find the optimal profit and the list of items using dynamic programming. **(10+10=20 marks)**

BUET · CSE 105

DSA

Dynamic Programming

LCS, MCM, Edit Distance, Knapsack, Memoization

T11 — Dynamic Programming

Q1. Assume that you have obtained a gold bar of integer length n. You plan to cut it into smaller pieces and sell them to the market. For any integer $i \leq n$, you know the profit you will obtain if you sell a gold bar of length i.

(a) Using the cut-and-paste method, formulate an argument to show that the problem of obtaining the maximum profit by cutting this gold bar into smaller pieces has the optimal substructure property.

(b) Construct a recursion tree diagram to show that the problem has the overlapping subproblem property.

(7+8=15 marks) [CSE 105 | 2023–24]

Q2. Given the two strings $X =$ ``AGGTAB'' and $Y =$ ``GXTYAB'', construct the dynamic programming (DP) table to compute the length of the Longest Common Subsequence (LCS) between X and Y. Clearly state the LCS and its length. **(15 marks)** [CSE 105 | 2023–24]

Q3. You are given a sequence of matrices $\langle A_1, A_2, \ldots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \ldots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to compute the product of the given chain of matrices. Formulate a recurrence relation to solve this problem. Design an efficient dynamic programming algorithm using the recurrence relation. **(15 marks)** [CSE 105 | 2023–24]

Q4. You are given a sequence of matrices $\langle A_1, A_2, \ldots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \ldots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to determine the product of the given chain of matrices.

(a) Formulate arguments to show that the MCM problem has the optimal substructure property.

(b) Compose a recursion tree diagram to show that MCM has the overlapping subproblem property.

(c) Design an efficient algorithm using the dynamic programming approach to solve this problem.

(8+7+10=25 marks) [CSE 105 | 2022–23]

Q5. Using a dynamic programming algorithm, determine the minimum edit distance alignment of the two strings $X =$ ATCGGT and $Y =$ ACCDT, where both the gap penalty and mismatch penalty are 1 and the cost of a match is 0. Show the DP table, the alignments, and mark the paths (which correspond to the alignments) in the DP table. **(10 marks)** [CSE 105 | 2022–23]

Q6. Given two strings, the longest common subsequence (LCS) problem is to find the longest subsequence present in both strings in the same relative order, but not necessarily contiguous. For example, for $S_1 =$ ``ALGORITHM'' and $S_2 =$ ``LITHIUM'', the LCS is ``LITHM''. Present a dynamic programming algorithm to find the length of the LCS of two strings. Derive a proof of correctness of the algorithm and also derive its running time. **(8+6+3=17 marks)** [CSE 105 | 2021–22]

Q7. Solve the following 0-1 knapsack problem instance (capacity = 6) using dynamic programming. Show the DP table and find the optimal set of items.

Item	Weight	Value
1	2	8
2	1	7
3	2	12
4	2	6
5	2	10

(15 marks) [CSE 203 | 2021–22]

Q8. Find the best alignment (minimum edit distance) of the following two DNA sequences using dynamic programming. Use gap penalty = 3, mismatch penalty = 2, match penalty = 0. Show the DP table, the alignment, and mark the corresponding path.

CCAGTGC vs CGATC

(15 marks) [CSE 203 | 2021–22]

Q9. Give a dynamic programming algorithm to find the length of the longest common subsequence (LCS) of two strings. (A common subsequence appears in both strings in the same relative order but not necessarily contiguously.) **(20 marks)** [CSE 203 | 2021–22]

Q10. Derive a recursive definition to find $m[i, j]$ (the minimum number of multiplications to compute $A_i \cdot A_{i+1} \cdots A_j$). With a recursion tree, describe the overlapping sub-problem property of Matrix Chain Multiplication (MCM). Then, given six matrices $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$, $A_6(20 \times 25)$ and the following partial DP table, give an ordering to populate the marked cells and calculate $m[2, 5]$:

	1	2	3	4	5	6
1	0	15750	7875	$m[1,4]$	$m[1,5]$	$m[1,6]$
2		0	2625	4375	$m[2,5]$	$m[2,6]$
3			0	750	2500	$m[3,6]$
4				0	1000	$m[4,6]$
5					0	$m[5,6]$
6						0

(7+6+10 marks) [CSE 203 | 2020–21]

Q11. At the beginning of the COVID-19 pandemic, two biologists were identifying the similarity between COVID-19 and SARS viruses using genome sequences $S'_1 = \langle A, T, T, G, C, A, T \rangle$ and $S'_2 = \langle T, A, G, C, C, T \rangle$.

(a) (Dr. Brown's task) Write the recursive definition for the Longest Common Subsequence (LCS) problem and populate the DP table with arrows showing how cell values propagate. Find the length and the LCS itself.

(b) (Dr. William's task) Write the recursive definition for the edit distance problem using operations {Insert (cost 1), Delete (cost 1), Replacement (cost 1), Keep (cost 0)}. Populate the DP table and show the edit distance cost and sequence of required operations.

(15 marks) [CSE 203 | 2020–21]

Q12. Discuss a problem scenario where adopting memoization over bottom-up dynamic programming can result in a faster solution. Justify your answer. **(5 marks)** [CSE 203 | 2020–21]

Q13. Apply dynamic programming to calculate the edit distance of the following two DNA sequences, where the cost of insertion or deletion is 2 and the cost of substitution is 1. Write the recurrence relation, explain the intuition behind it, populate the DP table, and calculate the edit distance.

`TGCATAT` vs `ATCCGA`

(15 marks) [CSE 203 | 2019–20]

Q14. Briefly explain the steps to be followed for solving a problem using dynamic programming. **(10 marks)** [CSE 203 | 2018–19]

Q15. Dream coins are used by the people of Dreamland. Only the following coins are available: \$1, \$7, \$11, and \$15.

(a) Write an example of an amount where the greedy strategy for the Coin-Change problem does *not* provide an optimal solution. Mention both the greedy and optimal solutions.

(b) Using dynamic programming, find the minimum number of coins that add up to \$20.

(10 marks) [CSE 203 | 2018–19]

Q16. Consider the following 0/1 knapsack problem instance (capacity = 7):

Item	Weight	Value
1	1	50
2	3	60
3	5	100
4	2	70
5	2	80

Solve this instance using dynamic programming. Show the DP table and find all solutions using the table. **(15 marks)** [CSE 203 | 2018–19]

Q17. Find all longest common subsequences of $X = \langle T, A, A, G, U, A, T, U \rangle$ and $Y = \langle A, U, A, G, T, U, T \rangle$ using dynamic programming. Also find a shortest common supersequence of X and Y using an LCS of X and Y. **(15 marks)** [CSE 203 | 2018–19]

Q18. Suppose you run a bus lending business, and each request has the format (start time, end time). Find all solutions for the maximum number of requests that can be satisfied by:

(a) Earliest start time

(b) Shortest interval

(c) Earliest end time

Use the following requests: $\{[1, 6), [4, 8), [6, 8), [9, 15), [6, 9), [11, 15), [2, 6)\}$.
(12 marks) [CSE 203 | 2018–19]

Q19. Let $A_1(11 \times 4)$, $A_2(4 \times 5)$, $A_3(5 \times 3)$, $A_4(3 \times 6)$, $A_5(6 \times 7)$, $A_6(7 \times 1)$ be six matrices. Compute $m[1, 6]$ given: $m[1, 3] = 35$, $m[2, 4] = 132$, $m[1, 5] = 95$, $m[2, 5] = 270$, $m[2, 6] = 95$, $m[4, 5] = 126$, $m[4, 6] = 60$. **(13 marks)** [CSE 203 | 2018–19]

Q20. Briefly explain with examples: (i) Optimal substructure, and (ii) Overlapping subproblems. Then recall the divide-and-conquer algorithm for merge sort and analyze its running time. **(8+7 marks)** [CSE 203 | 2017–18]

Q21. Consider the following instance of the 0-1 knapsack problem (capacity = 6):

Item	Weight	Value
1	2	7
2	1	8
3	2	12
4	2	6
5	2	10

Solve this instance using dynamic programming. Show the DP table and find the optimal set of items from the table. **(15 marks)** **[CSE 203 | 2017–18]**

Q22. Given two strings, the longest common *substring* problem is to find the longest string that is a substring of both given strings. For example, for $S_1 =$ `buetcserocks` and $S_2 =$ `bdcserocks`, the longest common substring is `cserocks`. Give a dynamic programming algorithm to find the length of the longest common substring. **(15 marks)** **[CSE 203 | 2017–18]**

Q23. Solve the following instance of the 0-1 knapsack problem using a branch-and-bound algorithm with a state-space tree (capacity = 10 kg):

Item	Weight (kg)	Profit (Taka)
1	3	18
2	5	35
3	4	44
4	7	36
5	2	18

(8 marks) **[CSE 207 | 2017–18]**

Q24. Recall the Weighted Interval Scheduling Problem. You are given n intervals with starting time s_i, finishing time f_i, and value v_i. A subset of intervals is compatible if no two overlap. Give a dynamic programming algorithm to find a compatible subset maximizing $\sum_{i \in S} v_i$. Your algorithm (including preprocessing) should run in $O(n \log n)$ time. **(15 marks)** **[CSE 203 | 2016–17]**

Q25. Consider the following Boolean formula: $(x' \vee y)(x \vee y' \vee z)(y \vee y')(x \vee y)(x \vee y \vee z')(x \vee y \vee z)$. Use a backtracking algorithm to decide whether it is satisfiable. Show the backtracking tree. **(15 marks)** **[CSE 207 | 2016–17]**

Q26. Solve the following 0-1 knapsack instance (capacity = 10) using the branch-and-bound algorithm. Show the branch-and-bound tree.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

(20 marks) **[CSE 207 | 2016–17]**

Q27. What is the matrix chain multiplication problem? Prove that it has both the overlapping subproblems property and the optimal substructure property. **(10 marks)** **[CSE 207 | 2015–16]**

Q28. Write the steps to be followed for solving a problem using dynamic programming. Let $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$ be sequences, and let $Z = (z_1, \ldots, z_k)$ be any LCS of X and Y. If $x_m \neq y_n$ and $z_k \neq x_m$, prove that Z is an LCS of X_{m-1} and Y. **(4+6=10 marks)** **[CSE 207 | 2015–16]**

Q29. Find all longest common subsequences of $X = \langle A, G, G, C, T, G, A, T \rangle$ and $Y = \langle G, G, G, C, A, T, A \rangle$ using dynamic programming. **(15 marks)** **[CSE 207 | 2015–16]**

Q30. Solve the following 0/1 knapsack instance (capacity = 15) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	140
2	6	132
3	4	140
4	3	90

Compare the backtracking and branch-and-bound techniques. **(3+12=15 marks)** **[CSE 207 | 2015–16]**

Q31. What are the key properties of an optimization problem that allow it to be solved using dynamic programming? Compare top-down memoized DP with bottom-up DP, and identify types of problems for which one outperforms the other. **(12 marks)** **[CSE 207 | 2014–15]**

Q32. The balanced partition problem: given n integers each in $[0, N]$, divide them into two subsets minimizing the difference of their sums (e.g., for $S = \{1, 7, 3, 5, 9\}$, a balanced partition is $S_1 = \{1, 7, 5\}$ and $S_2 = \{3, 9\}$ with difference 1). Formulate a dynamic programming solution and determine its time complexity. **(12 marks)** [CSE 207 | 2014–15]

Q33. Using dynamic programming, compute the minimum number of scalar multiplications for the matrix chain multiplication of $A_1, A_2, A_3, A_4, A_5, A_6$ with dimensions 15×5, 5×50, 50×20, 20×10, 10×35, 35×25. Also show the ordering of the matrices for the minimum. **(15 marks)** [CSE 207 | 2013–14]

Q34. Compare dynamic programming and memoization techniques. "A dynamic programming algorithm usually outperforms the corresponding memoized algorithm" — explain. **(10 marks)** [CSE 207 | 2013–14]

Q35. Write a memoized algorithm for solving the matrix chain multiplication problem. Analyze its time complexity. **(10 marks)** [CSE 207 | 2013–14]

Q36. What is a state-space tree? Solve the following instance of the 0/1 knapsack problem (capacity = 10) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	42
2	4	40
3	3	12
4	5	25

(15 marks) [CSE 207 | 2013–14]

Q37. Prove that the longest common subsequence problem has both the Overlapping Subproblems property and the Optimal Substructure property. **(10 marks)** [CSE 207 | 2013–14]

Q38. Find all longest common subsequences of $X = \langle A, T, G, C, T, G, A, T\rangle$ and $Y = \langle T, G, G, C, A, T, A\rangle$ using dynamic programming. **(15 marks)** [CSE 207 | 2013–14]

Q39. In a previous life, you worked as a cashier in the ancient city of Egypt, spending the better part of your day giving change to your customers. Because paper was a very rare and valuable resource at the time, cashiers were required by law to use the fewest notes possible whenever they gave change. The currency of Egypt at that time, called Dream Dollars, was available in the following denominations: \$1, \$4, \$7, \$13, \$28, \$52, \$91, \$365. **(20 marks)**

(a) A greedy change algorithm repeatedly takes the largest note that does not exceed the target amount. For example, to make \$122 using the greedy algorithm, we first take a \$91 note, then a \$28 note, and finally three \$1 notes. Give an example where this greedy algorithm uses more Dream Dollar notes than the minimum possible.

(b) Describe a dynamic programming algorithm that computes, given an integer k, the minimum number of notes needed to make k Dream Dollars. Write down the pseudocodes for both the memoized approach and bottom-up iterative approach. Analyze the running-times of both approaches.

[CSE 207 | 2012–13]

Q40. You have n magic boxes kept in a row, numbered 1 to n, each containing some dollars. You can open box i only if you did not open box $i-1$ or box $i-2$. Given the amount $D[1 \ldots n]$ in each box, find the maximum dollars obtainable. Describe a dynamic programming solution and analyze its running time. **(12 marks)** [CSE 207 | 2012–13]

Q41. Suppose we are given a sequence of integers $A[1..n]$ and we want to find the longest subsequence whose elements are in decreasing order. More concretely, we want to find the longest sequence of indices $1 < i_1 < i_2 < \ldots < i_k < n$ such that $A[i_j] > A[i_{j+1}]$ for all j. Describe the pseudocode of a backtracking algorithm that solves this problem in $O(2^n)$ time. **(15 marks)** [CSE 207 | 2012–13]

Q42. Compare the properties of problems that can be solved with the greedy method, divide-and-conquer method, and dynamic programming method. **(3 marks)** [CSE 207 | 2011–12]

Q43. Write and prove the optimal-substructure property of the matrix chain multiplication problem. Calculate the number of parenthesizations of a sequence of n matrices. **(5+5=10 marks)** [CSE 207 | 2011–12]

Q44. Find an optimal solution to the 0/1 knapsack instance of $n = 4$, $W = 6$, $(v_1, v_2, v_3, v_4) = (50, 30, 12, 45)$, $(w_1, w_2, w_3, w_4) = (2, 3, 1, 3)$. **(15 marks)** [CSE 207 | 2011–12]

Q45. Let $X = (x_1, x_2, \ldots, x_m)$ and $Y = (y_1, y_2, \ldots, y_n)$ be sequences, and let $Z = (z_1, z_2, \ldots, z_k)$ be any LCS of X and Y. If $x_m \neq y_n$ and $z_k \neq x_m$, prove that Z is an LCS of X_{m-1} and Y. **(2 marks)** [CSE 207 | 2011–12]

Q46. Find an LCS of $X = \langle A, T, C, T, G, A, T\rangle$ and $Y = \langle T, G, C, A, T, A\rangle$ by showing the c and b tables. **(10 marks)** [CSE 207 | 2011–12]

Q47. Give an example of a problem that has the optimal substructure property. Give another example of a problem that does not have optimal substructure. Justify your answers. **(10 marks)** [CSE 207 | 2009–10]

Q48. Show the execution of the Dynamic Programming LCS algorithm for the following two strings: $X = \langle A, B, C, B \rangle$ and $Y = \langle B, D, A, C \rangle$. **(15 marks)** [CSE 207 | 2009–10]

Q49. Show all the different ways to multiply the matrix chain $A_1A_2A_3A_4A_5$. **(5 marks)** [CSE 207 | 2009–10]

Q50. Convert the following recursive procedure for computing the n-th Fibonacci number into an equivalent procedure that uses a dynamic multi-graph representing the computation. From this graph, find $T_1(5)$, $T_\infty(5)$, and the parallelism.

```
F(n)
  if n <= 1 then return n
  else return F(n-1) + F(n-2)
```

(17 marks) [CSE 207 | 2009–10]

Q51. Find a longest common subsequence from the DNA sequences `ACGCTAC` and `CTGCA`. Print the sequence also. Analyze the run times of your methods of finding and printing the sequence. **(10+5=15 marks)** [CSE 207 | 2008–09]

Q52. Find the optimal sequence of multiplying the following matrices: $A(2 \times 7)$, $B(7 \times 9)$, $C(9 \times 4)$, $D(4 \times 6)$, $E(6 \times 3)$. How does the memoization technique help? **(10+5=15 marks)** [CSE 207 | 2008–09, 2006–07]

Q53. Mention the basic properties that must be satisfied to solve a problem using dynamic programming. When and how can memoization help improve an exponential-time recursive algorithm to polynomial time? **(7+8 marks)** [CSE 207 | 2007–08]

Q54. Suppose a thief went to a superstore to steal. He has a bag with him which can contain at most W unit of weight of anything. The thief can carry at most W' unit of weight. In the store there are two types of things. For Type 1 things, he either has to take the thing or leave it. For Type 2 things, he can weigh and take as per his will (e.g. sugar, rice, salt etc.). All type 1 things are labeled with the corresponding unit prices and each type 2 thing is labeled with the corresponding price per unit weight. The thief decided to take only the things belonging to Type 1. He now needs an algorithm to maximize his profits.

(a) Map the above problem to one of your known problems. **(5 marks)**

(b) Discuss possible greedy strategies to solve the problem and discuss whether or not any of them will work. **(7 marks)**

(c) If the thief decides to take only type 2 things, will any of the greedy strategies work? Justify your answer. **(6 marks)**

[CSE 207 | 2007–08]

Q55. Find an optimal solution to the 0–1 knapsack problem with the following weight and profit values (weight limit = 8): $(1, 2)$, $(1, 3)$, $(2, 3)$, $(2, 5)$, $(3, 4)$ **(10 marks)** [CSE 207 | 2006–07]

Q56. Construct four matrices for finding all-pairs shortest paths for a given graph (edges: $AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), F$ $CE(4), DE(6), FD(3)$). **(15 marks)** [CSE 207 | 2006–07]

Q57. Find the longest common subsequence from the DNA sequences `ACGCTAC` and `CTGCA`. Print the LCS and analyze the runtime of both finding and printing the sequence. **(10+5 marks)** [CSE 207 | 2006–07]

Q58. Find a $\Theta(n^2 2^n)$ TSP tour for the following distance matrix. What is the major drawback of this solution method?

	A	B	C	D
A	0	10	15	20
B	5	0	9	10
C	6	13	0	12
D	8	8	9	0

(12 marks) [CSE 207 | 2006–07]

BUET · CSE 105

DSA

Comparison-Based Sorting

Quicksort, Mergesort, Heapsort, Insertion Sort

T12 — Comparison-Based Sorting

Q1. Analyze the worst-case runtime of the Quicksort algorithm under the following assumptions. In your analysis, formulate the worst-case runtime as a recurrence relation and solve the recurrence.

(a) Balanced partitioning

(b) Imbalanced partitioning

(5+5=10 marks) [CSE 105 | 2023–24]

Q2. Define stable sorting algorithm. Explain with an example. **(5 marks)** [CSE 105 | 2023–24]

Q3. You are asked to sort the following array of integers in ascending order using the Quicksort algorithm. Highlight the pivot item at each step and show the status of the array after processing each pivot element:

$$[5,\ 17,\ 2,\ 4,\ 16,\ 8,\ 10,\ 6,\ 4,\ 26,\ 20,\ 7]$$

(15 marks) [CSE 105 | 2022–23]

Q4. Quick-Sort is a divide-and-conquer based algorithm. The traditional Partition function of Quick-Sort works by selecting a *pivot* element from the input array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the *pivot*. For this reason, it is sometimes called *partition-exchange* sort. The sub-arrays are then sorted recursively. This can be done *in-place*, requiring small additional amounts of memory to perform the sorting. Now answer the following questions.

(i) What is the worst-case running time of Quick-Sort algorithm if it is designed in a way that the last element of the input array is always selected as the *pivot*? Also give an example input scenario which will bring about the worst-case running time.

(ii) Suppose, you have been asked to evaluate the performance of Quick-Sort algorithm. You are using a bad input array which results in a very unbalanced partition at each of the recursive steps. Consider that at each recursive call, the Partition function is producing an α-to-β proportional split. Now using a recursion tree, find the time complexity of the Quick-Sort algorithm for this scenario. If the last three digits of your student ID represents a number x, then consider the following:

$$\alpha = x \quad \text{if } x \leq 100$$
$$\alpha = 200 - x \quad \text{if } x > 100$$
$$\beta = 100 - \alpha$$

Your answer must contain a well-drawn recursion tree where any kind of existing skewness of the tree is clearly illustrated.

(5+7=12 marks) [CSE 203 | 2020–21]

Q5. Apply the partition procedure of Quicksort once on the array below, where the pivot is chosen as the element at the middle index $\lfloor (p+q)/2 \rfloor$ (indices range from p to q inclusive). Show all steps.

$$[10,\ 30,\ 50,\ 70,\ 90,\ 100,\ 80,\ 60,\ 40,\ 20]$$

(10+7 marks) [CSE 203 | 2019–20]

Q6. Describe a scenario where an in-place sorting algorithm is necessary and explain the reasons. **(5 marks)** [CSE 203 | 2019–20]

Q7. Give a concise description of a good way for Quicksort to choose a pivot element (better than always using index 0). **(5 marks)** [CSE 203 | 2018–19]

Q8. Write down the Quicksort algorithm. Use the partition algorithm to find the smallest 5 values of the array $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$, showing every step. **(5+10 marks)** [CSE 203 | 2017–18]

Q9. What is an in-place sorting algorithm? Write an in-place sorting algorithm. Analyze the worst-case and best-case time complexities of the algorithm. **(15 marks)** [CSE 203 | 2016–17, 2015–16]

Q10. Derive the expected time complexity of randomized Quicksort. **(20 marks)** [CSE 207 | 2016–17, 2009–10]

Q11. Write the Insertion Sort algorithm. Analyze the worst-case and best-case time complexities of the algorithm. **(10 marks)** [CSE 203 | 2015–16]

Q12. Write the Quick Sort algorithm. Analyze the best-case and worst-case time complexity of the algorithm. **(15 marks)** [CSE 203 | 2014–15, 2013–14]

Q13. Write the pseudocode of the Quicksort algorithm including the partition procedure. Deduce its average-case complexity. Simulate the partition algorithm on the array $[6, 2, 7, 3, 8, 4, 9, 5, 10, 1, 11]$ using the first element as the partitioning element. Show whenever elements are swapped. **(10 marks)** [CSE 207 | 2013–14]

Q14. Write a sorting algorithm that can always sort n elements in $O(n \log n)$ time. Analyze the best-case and worst-case time complexity of the algorithm. **(7+8=15 marks)** [CSE 203 | 2013–14]

Q15. Explain the divide-and-conquer technique. Write the quick sort algorithm and analyze the time complexity of the algorithm. **(15 marks)** **[CSE 203 | 2013–14]**

Q16. Sort the following dataset using Quicksort in *descending* order, using the first element as pivot (show all steps):

$$D = \{6,\ 0,\ 2,\ 0,\ 1,\ 3,\ 6,\ 1\}$$

(12 marks) **[CSE 203 | 2012–13]**

Q17. Using Mergesort on the dataset $D = \{6, 0, 2, 0, 1, 3, 6, 1\}$, draw the merge tree. Show that Mergesort runs in time $\Theta(n \log n)$ in the worst case. **(6+4=10 marks)** **[CSE 203 | 2012–13]**

Q18. Write the Quicksort algorithm. Simulate the Quicksort algorithm on the array

$$[15,\ 13,\ 9,\ 5,\ 12,\ 8,\ 7,\ 4,\ 10,\ 6,\ 2,\ 1,\ 26,\ 23]$$

showing in detail how the partition routine works. **(8+7=15 marks)** **[CSE 203 | 2011–12]**

Q19. Simulate the Quicksort algorithm on the array $[5, 1, 6, 3, 7, 8, 9, 2, 4, 8]$. Discuss its worst-case complexity. **(15 marks)** **[CSE 207 | 2008–09]**

BUET · CSE 105

DSA

Sorting in Linear Time

Counting Sort, Radix Sort, Bucket Sort

T13 — Sorting in Linear Time

Q1. Sort the following list of non-negative integers using Radix Sort (LSD):

$$[329,\ 457,\ 657,\ 839,\ 436,\ 720,\ 355]$$

State the conditions required for Radix Sort to work correctly and in linear time. **(10 marks)** [CSE 105 | 2023–24]

Q2. Given two arrays $a[\,]$ and $b[\,]$, each containing N distinct points in the plane, design an algorithm with linear running time (under reasonable assumptions) and at most linear extra space that determines whether the two arrays contain precisely the same set of points. **(8 marks)** [CSE 203 | 2018–19]

Q3. Write a linear-time Bucket Sort algorithm. Prove that the time complexity of the algorithm is linear. **(10 marks)** [CSE 203 | 2014–15, 2011–12]

Q4. Write the counting sort algorithm. Prove that the algorithm runs in linear time. Explain why counting sort is stable. **(12 marks)** [CSE 203 | 2013–14]

Q5. Write the Counting Sort steps for the dataset $D = \{6, 0, 2, 0, 1, 3, 6, 1\}$. **(6 marks)** [CSE 203 | 2012–13]

Q6. Sort the following dataset using Counting Sort, showing every step:

$$A = \{6,\ 0,\ 2,\ 0,\ 1,\ 3,\ 4,\ 6,\ 1,\ 3,\ 2\}$$

(11 marks) [CSE 203 | 2011–12]

Q7. "An important property of Counting Sort is that it is stable" — explain. **(4 marks)** [CSE 203 | 2011–12]

BUET · CSE 105

DSA

Lower Bound Theory

Comparison Sort Lower Bound, Decision Trees

T14 — Lower Bound Theory

Q1. Prove that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. **(10 marks)** [CSE 105 | 2023–24, 2022–23 , 2022-23, CSE 203 | 2016–17, 2014–15, 2013–14, CSE 207 | 2009–10]

Q2. Prove that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. Describe a sorting algorithm that achieves this lower bound. **(7+8 marks)** [CSE 203 | 2016–17]

BUET · CSE 105

DSA

Data Structures for Set Operations

Union-Find, Disjoint Sets, Path Compression, Union by Rank

T15 — Data Structures for Set Operations

Q1. Consider a Union-Find data structure on a set of n elements, where the UNION operation keeps the name of the largest set. Analyze the run-time complexity of any sequence of k UNION operations. **(10 marks)** [CSE 105 | 2023–24, 2022–23]

Q2. Write an algorithm to detect a cycle in an undirected graph using a disjoint-set data structure. Mention the cases where this algorithm performs more efficiently than BFS- or DFS-based approaches. **(10+5 marks)** [CSE 203 | 2019–20]

Q3. Explain, with examples, the union-by-rank and path-compression heuristics used in the UNION and FIND operations of disjoint-set data structures. **(10 marks)** [CSE 203 | 2018–19]

Q4. We have a Union-Find data structure for a set S of size n, where unions keep the name of the larger set. Prove that for the array implementation, any sequence of k UNION operations takes at most $O(k \log k)$ time. Also prove that for the pointer-based implementation, a FIND operation takes $O(\log n)$ time. **(15 marks)** [CSE 207 | 2011–12]

Q5. What is the Disjoint Set forest implementation? Write short notes on path compression. Write pseudocode for Disjoint Set forest with path compression. **(3+3+4 marks)** [CSE 207 | 2009–10]